

Machine Learning I

Digitization and Data Analytics — TU Dresden

Comprehensive Master-Level Study Guide

Similarity & Distance Measures · Clustering (Partitioning, Hierarchical, Density-Based) · Cluster Assessment ·
Big-Data Clustering

TU Dresden · MS Computer Science

Based on Klumpke, CIDS/ZIH Lecture Slides

Exam / Research-Oriented Q&A Included

Table of Contents

1. Part I — Foundations of Machine Learning
 - a. The Data Analytics Pipeline
 - b. What is Machine Learning? Two Perspectives
 - c. Taxonomy: Supervised / Unsupervised / Semi-Supervised / Reinforcement
 - d. Supervised Learning Formalism & Classification
 - e. Unsupervised Learning Formalism
 - f. Part I Q&A Bank
2. Part II — Similarity and Distance Measures
 - a. Why the Choice of Distance Matters
 - b. Similarity vs. Dissimilarity
 - c. Minkowski (L_p) Distances for Quantitative Data — Full Worked Example
 - d. Categorical Data Similarity (Simple Matching & Inverse Occurrence Frequency)
 - e. Text Data — Cosine Similarity
 - f. Discrete Sequences — Edit (Levenshtein) Distance & LCSS
 - g. Graph Similarity Measures
 - h. Time Series — Dynamic Time Warping (DTW)
 - i. High-Dimensional Data & the Curse of Dimensionality
 - j. Part II Q&A Bank
3. Part III — Clustering
 - a. Definitions, Applications & Domains
 - b. Design Considerations for Clustering
 - c. Cluster Quality: Preconditions & Goals
 - d. Overview of Major Clustering Families
 - e. Partitioning Methods: k-Means
 - f. Partitioning Methods: k-Medoids / PAM
 - g. Hierarchical Clustering: AGNES & DIANA, Linkage Criteria
 - h. Density-Based Clustering: DBSCAN
 - i. Cluster Assessment (Hopkins Statistic, Elbow, Silhouette, External Validation)
 - j. Aspects of Big-Data Clustering & Complexity
 - k. Part III Q&A Bank
4. Part IV — Comprehensive Exam & Research-Oriented Question Bank (30 Questions)
5. Part V — Cheat Sheet: Formulas, Algorithms & Comparison Tables

Foundations of Machine Learning

1.1 The Data Analytics Pipeline

Data analytics is not a single algorithm — it is a **cyclical process**. TU Dresden's lecture frames it as four stages that feed back into one another: **Data Collection** → **Data Preprocessing** (extraction, cleaning, reduction, annotation, aggregation) → **Data Analysis** (prediction, exploration, modeling) → **Interpretation**, whose findings loop back to guide further collection.

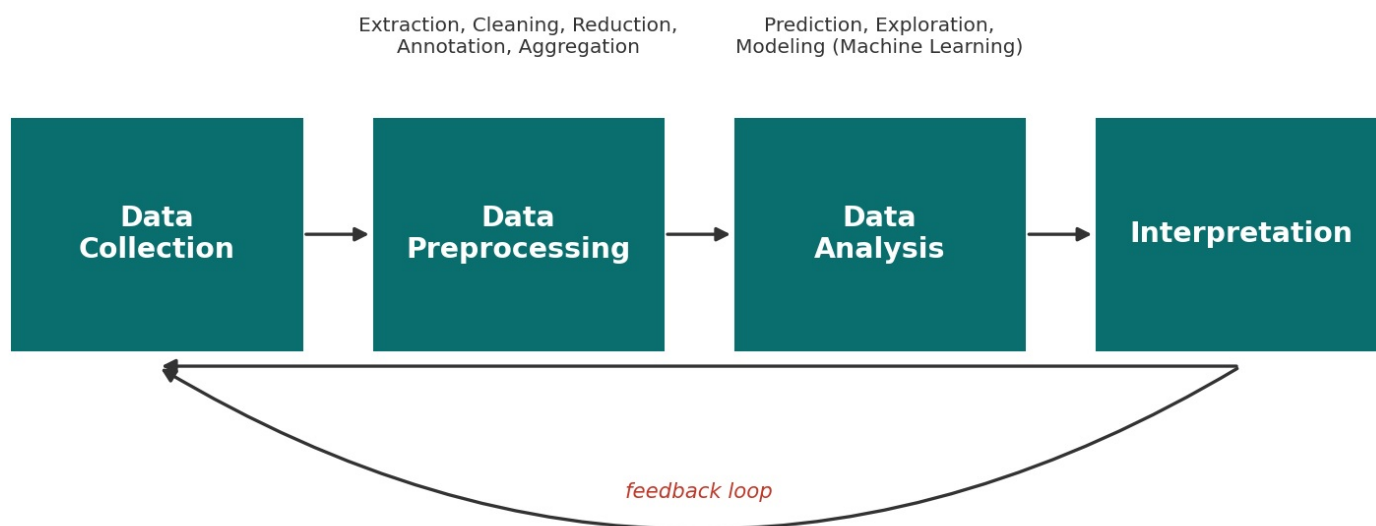


Fig. 1.1 — The Data Processing & Analytics cycle. Machine Learning lives inside the "Data Analysis" box, automating the modeling/prediction/exploration step.

KEY INSIGHT

Machine Learning is best understood as the **automated engine** that powers the "Data Analysis" stage once data volumes get too large for manual analysis. This is why the slides insert "Machine Learning" as an arrow pointing directly into the Data Analysis box.

1.2 What is Machine Learning? Two Complementary Perspectives

(a) The Classical "Learning" Perspective — Tom Mitchell (1997)

A computer program is said to **learn from experience E** with respect to some class of **tasks T** and **performance measure P**, if its performance at tasks in T, as measured by P, **improves with experience E**.

This definition is operational: to claim "learning" you must be able to name T (the task, e.g. classifying flowers), P (the metric, e.g. accuracy), and E (the experience, e.g. labeled training examples). It is the definition examiners most often ask you to *apply* to a concrete scenario (see Q&A).

(b) The Probabilistic Perspective — Kevin Murphy (2022)

"...we define machine learning as a set of methods that can automatically **detect patterns** in data, and then use the uncovered patterns to **predict future data**, or to perform other kinds of **decision making under uncertainty**."

Murphy's probabilistic framing treats every unknown quantity — future observations, model parameters — as a **random variable** endowed with a probability distribution. This is the foundation for later lectures on Bayesian learning, and it explains why supervised learning is often written as fitting $p(y|x)$ and unsupervised learning as fitting $p(x)$ (see below).

EXAM TIP

If asked "define machine learning," give *both* definitions and explicitly connect them: Mitchell's is behavioral/operational (how do we know learning happened?), Murphy's is representational (what does the learned object look like — a probability distribution).

1.3 Taxonomy of Machine Learning

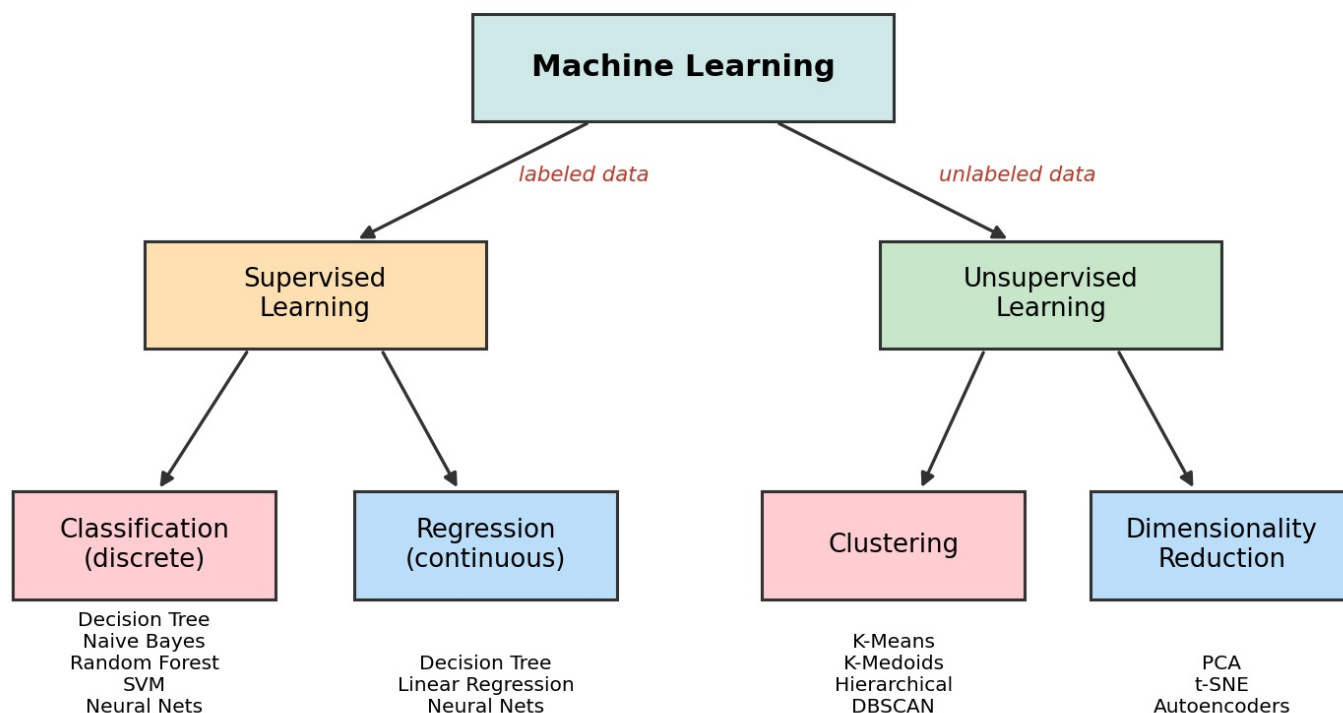


Fig. 1.2 — The complete ML taxonomy used in this course: the labeled/unlabeled split determines supervised vs. unsupervised learning; the discrete/continuous split of the target determines classification vs. regression (supervised) or clustering vs. dimensionality reduction (unsupervised).

Type	Data	Guiding Question	Goal	Typical Methods
Supervised	Labeled: $(\tilde{x}, \tilde{y})$ pairs	"What could happen?"	Learn $f: \tilde{x} \rightarrow \tilde{y}$ to predict the future	Decision Trees, Naive Bayes, Random Forest, SVM, Neural Nets, Linear Regression
Unsupervised	Unlabeled: $\tilde{x}$ only	"What has happened?"	Discover hidden structure / interesting patterns	k-Means, Hierarchical Clustering, DBSCAN, PCA
Semi-Supervised	Mix of labeled & unlabeled	Bridge of the two above	Exploit abundant unlabeled + scarce labeled data	Self-training, graph-based methods, consistency regularization
Reinforcement	Reward/punishment signals	"What should I do next?"	Learn a policy maximizing cumulative reward	Q-Learning, Policy Gradient (out of scope of this lecture series)

NOT COVERED FURTHER IN THIS COURSE

Semi-supervised and reinforcement learning are introduced only for completeness in this lecture (Chapelle et al. 2006; Sutton & Barto 1998) — the slides explicitly state they are *not* the topic of the remaining lecture series, which instead concentrates on unsupervised clustering.

1.4 Supervised Learning — Formal Definition

Given: a training set $D = \{(\tilde{x}_j, \tilde{y}_j)\}_{j=1}^N$ of N labeled input-output pairs, where

- $\tilde{x}$ is the **feature vector** (attributes, covariates, predictors), components x_i
- $\tilde{y}$ is the **output/label vector** (response variable, target), components y_i

Goal: learn a mapping $f: \tilde{x} \rightarrow \tilde{y}$ that generalizes to unseen inputs.

Informal View

- Describes properties of *future* data
- Question: "What could happen?"
- Applications: forecasting, scoring, classification of unknown data

Probabilistic View

Fitting a **conditional model** $p(y|x)$ — a distribution over outputs given inputs. Classification = discrete y ; Regression = continuous y .

WORKED EXAMPLE — IRIS FLOWER DATA SET (FISHER, 1936)

Four numeric features (Sepal.Length, Sepal.Width, Petal.Length, Petal.Width) plus a label $\text{Species} \in \{\text{setosa}, \text{versicolor}, \text{virginica}\}$. A decision tree learned on this labeled data might read: "If $\text{Petal.Length} < 2.5 \rightarrow \text{Setosa}$; else if $\text{Petal.Length} < 4.9 \rightarrow \text{Virginica}$; else if $\text{Petal.Width} \geq 1.6 \rightarrow \text{Virginica}$, else $\rightarrow \text{Versicolor}$." This is a classifier — its purpose is to label **new, unseen** flowers.

1.5 Unsupervised Learning — Formal Definition

Given: a training set $D = \{(\tilde{x}_j)\}_{j=1}^N$ — inputs only, **no labels**.

Goal: find interesting patterns — e.g., group data points $\tilde{x}_j$ based on an application-dependent distance measure.

Informal View

- Describes *existing* data
- Question: "What has happened?"
- Goal: extract potential patterns

Probabilistic View

Fitting an **unconditional model $p(\mathbf{x})$** that can generate new data $\mathbf{x}$. The central open challenge: *what kind of pattern is interesting*, and *how do we assess the extracted pattern* without ground-truth labels? (→ Part III.9, Cluster Assessment)

CRITICAL DISTINCTION — FREQUENTLY TESTED

When k-Means is run on the (unlabeled) Iris measurements, the resulting 3 clusters correlate strongly with the true species — **but the algorithm never sees, and does not output, any semantic label**. The clustering model has **no notion** of "setosa." Assigning meaning to clusters is a manual, post-hoc interpretation step performed by the analyst, not by the algorithm. Exam questions frequently probe this "clusters $\neq$ labels" distinction.

1.6 Part I Question & Answer Bank

Q1.1 State Tom Mitchell's definition of machine learning and apply it to a spam-filter scenario.

Answer: "A computer program learns from experience E w.r.t. task T and performance measure P if its performance at T , as measured by P , improves with E ." Applied: **T** = classifying emails as spam/not-spam; **P** = fraction of emails correctly classified (accuracy); **E** = a database of emails that have been hand-labeled spam/not-spam, used repeatedly to adjust the classifier's parameters.

Q1.2 Why does the slide deck insert an explicit "Machine Learning" arrow feeding into the "Data Analysis" box of the pipeline rather than treating ML as a separate, disconnected topic?

Answer: Because ML is not a goal in itself — it is the concrete mechanism by which the abstract "Data Analysis" stage (prediction, exploration, modeling) is carried out at scale. Manual analysis does not scale to big data; ML automates pattern detection so the pipeline can process large data volumes, which then feed into "Interpretation" and loop back to further data collection.

Q1.3 Distinguish "similarity" as used generally in data analytics from the narrower, everyday meaning of the word.

Answer: Formally, given two objects O_1, O_2 , similarity is a *numerical value* $\text{Sim}(O_1, O_2)$ (or dually a distance/dissimilarity $\text{Dist}(O_1, O_2)$) computed by an application-specific function — it need not match intuitive human notions of "alike." The classic slide example: is *green* more similar to *blue* or to *red*? Under one representation (visible-spectrum wavelength) green is closer to blue; under another (the position of the letters in the word "green"/"blue"/"red" in the alphabet-indexed string) a completely different answer can emerge. The point: **similarity is representation- and application-dependent**, not an objective absolute.

Q1.4 A dataset has continuous, unlabeled sensor readings and you are asked to "find groups of similar sensors." Classify this task within the ML taxonomy and justify. What if instead you are told which sensors are "faulty" vs "healthy" during training?

Answer: Unlabeled + goal is grouping → **unsupervised learning**, specifically **clustering** (discrete groups desired, not a continuous reduced representation, ruling out dimensionality reduction). If instead "faulty/healthy" labels are available during training and the goal is to predict this discrete label for new sensors → **supervised classification**.

Q1.5 [Research-oriented] Why is semi-supervised learning practically important, and what is its core assumption relative to purely supervised learning?

Answer: Labeled data is expensive (requires human annotation) while unlabeled data is often abundant and cheap. Semi-supervised learning tries to exploit the structure (e.g., cluster assumption / manifold assumption) present in the large unlabeled set to improve a model trained on only a few labeled examples — effectively using unsupervised structure discovery to regularize a supervised objective. (Chapelle, Schölkopf & Zien, 2006).

Similarity and Distance Measures

2.1 Why the Choice of Distance Function Matters

"Poor choices of the distance function can sometimes be disastrously misleading depending on the application domain." — Charu C. Aggarwal, *Data Mining: The Textbook*

Every downstream ML method in this course — k-Means, k-Medoids, AGNES/DIANA, DBSCAN — is built **on top of** a similarity/distance function. If that function does not reflect the true semantics of the application domain, the "patterns" discovered are artifacts of the metric, not of the data.

2.2 Similarity vs. Dissimilarity — Formal Definitions

Formal statement: Given two objects O1 and O2, determine a value of similarity $\text{Sim}(O1, O2)$ (or distance/dissimilarity $\text{Dist}(O1, O2)$) between them. Possible objects: images, text documents, time series, mixed/tabular data, graphs, sequences, etc.

	Similarity	Dissimilarity (e.g. distance)
Meaning	Numerical measure of how <i>alike</i> two objects are	Numerical measure of how <i>different</i> two objects are
Direction	Higher value $\Rightarrow$ more alike	Lower value $\Rightarrow$ more alike
Typical range	Often [0, 1]	Minimum often 0; upper bound varies (unbounded for many metrics)

2.3 Minkowski (L_p) Distances for Quantitative Data

For quantitative feature vectors $\vec{x} = (x_1, \dots, x_d)$, the general **Minkowski distance** (L_p -norm) is:

$$\text{Dist}(\vec{x}, \vec{y}) = (\sum_{i=1}^d |x_i - y_i|^p)^{1/p}$$

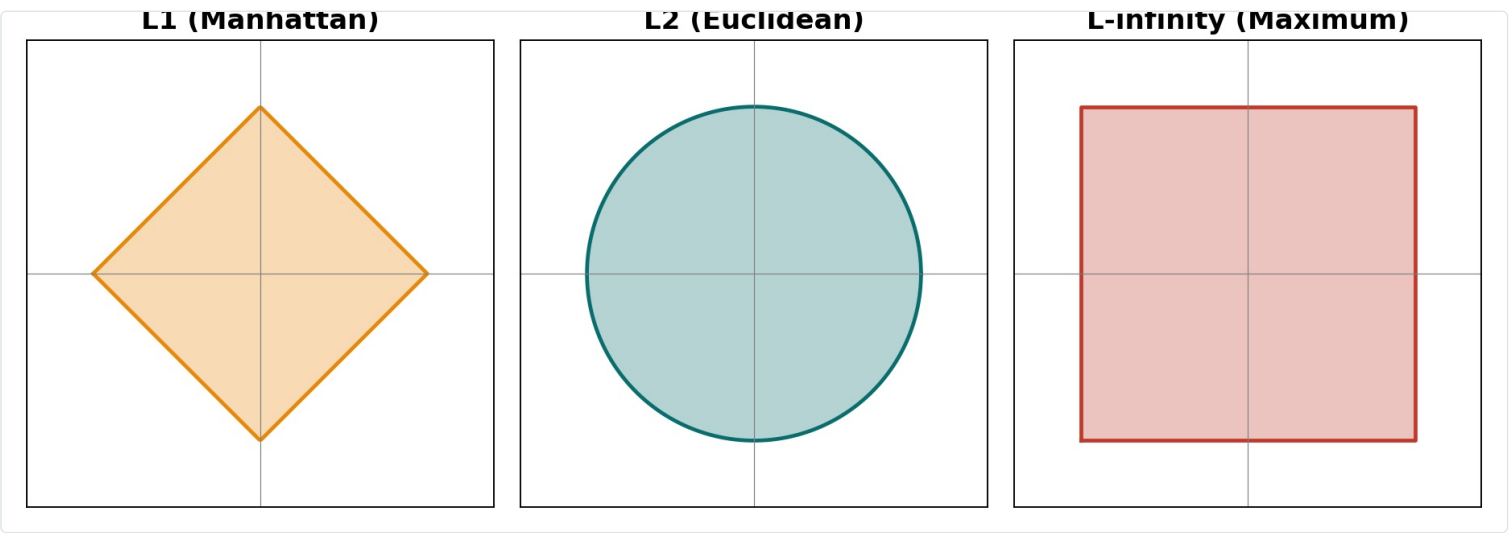


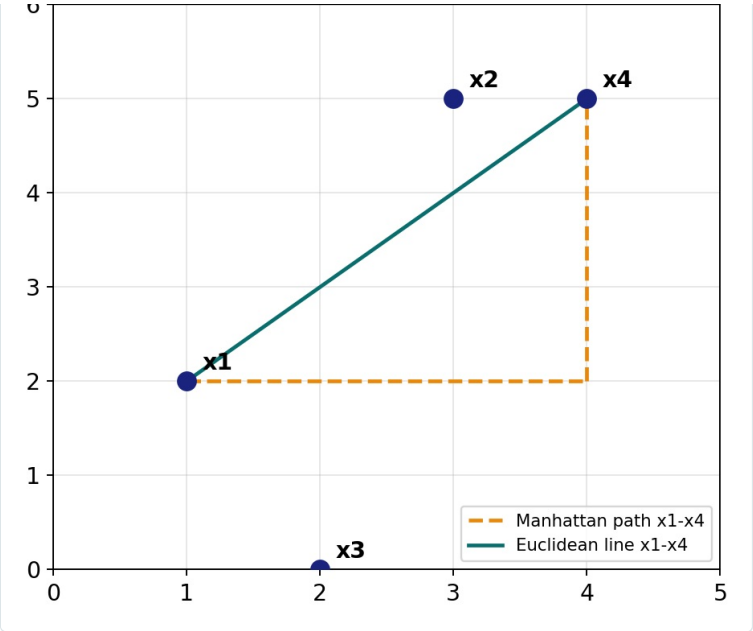
Fig. 2.1 — Unit "balls" (all points at distance 1 from the origin) for the three most important special cases of the Minkowski distance.

Name	p	Formula	Geometric meaning / when to use
Manhattan (L1)	1	$\sum x_i - y_i $	Sum of absolute per-axis differences ("city-block" / grid path). Robust to outliers on individual axes; used when movement is axis-constrained (e.g. street grids).
Euclidean (L2)	2	$\sqrt{\sum (x_i - y_i)^2}$	Straight-line distance. The default choice; rotation-invariant; used by classical k-Means.
Maximum / Chebyshev (L ∞)	∞	$\max_i x_i - y_i $	Only the single largest coordinate difference matters — e.g. chessboard king-move distance.

Full Worked Example (as used in the lecture)

	attribute 1	attribute 2	Example data points (attribute 1 vs attribute 2)
--	-------------	-------------	--

point		
x1	1	2
x2	3	5
x3	2	0
x4	4	5



Manhattan (L1) distance matrix

L1	x1	x2	x3	x4
x1	0			
x2	5	0		
x3	3	6	0	
x4	6	1	7	0

Euclidean (L2) distance matrix

L2	x1	x2	x3	x4
x1	0			
x2	3.61	0		
x3	2.24	5.10	0	
x4	4.24	1.00	5.39	0

Maximum (L∞) distance matrix

L∞	x1	x2	x3	x4
x1	0			
x2	3	0		
x3	2	5	0	
x4	3	1	5	0

FULLY WORKED CALCULATION: DIST(X1, X4)

$x1=(1,2)$, $x4=(4,5) \Rightarrow |\Delta a1|=3, |\Delta a2|=3$.
 $L1 = 3+3 = \mathbf{6}$. $L2 = \sqrt{(3^2+3^2)} = \sqrt{18} \approx \mathbf{4.24}$. $L\infty = \max(3,3) = \mathbf{3}$.
 Note the general inequality that always holds: $L\infty \leq L2 \leq L1$ (equality only when at most one coordinate differs).

2.4 Categorical Data Similarity

Categorical attributes have **no ordering**, so subtraction is meaningless. Similarity is instead built up per-attribute and summed: $S(\vec{x}, \vec{y}) = \sum_{i=1}^d S(x_i, y_i)$.

(a) Simplest matching similarity

$$S(x_i, y_i) = 1 \text{ if } x_i = y_i; \quad 0 \text{ otherwise}$$

Treats every mismatch equally, ignoring how common/rare a value is.

(b) Inverse Occurrence Frequency (accounts for rarity)

$$p_k(x) = (\text{ \# datapoints } \vec{x} \text{ with kth attribute} = x \text{ }) / (\text{ \# datapoints } \vec{x} \text{ }) \quad S(x_i, y_i) = 1/p_k(x_i)^2 \text{ if } x_i=y_i, \text{ else } 0$$

WORKED EXAMPLE – "ADULT" ATTRIBUTE

Table of 6 people: Adult=1 appears in 4/6 rows, Adult=0 in 2/6 rows $\Rightarrow p_{Adult}(0) = 1/3$, $p_{Adult}(1) = 2/3$. Two "Adult=0" people match

with weight $1/(1/3)^2 = 9$; two "Adult=1" people match with weight only $1/(2/3)^2 = 2.25$. **Rarer shared values contribute a much higher similarity score** — matching on a rare attribute value is more informative than matching on a common one (directly analogous to IDF weighting in text retrieval, §2.5).

2.5 Text Data — Cosine Similarity

Documents are represented as term-frequency vectors over a shared vocabulary. Cosine similarity computes the **angle** between two document vectors, making it insensitive to document length (a repeated document scaled $\times 10$ still points in the same direction):

$$\cos(\vec{x}, \vec{y}) = (\sum_{i=1}^d x_i y_i) / (\sqrt{\sum x_i^2} \cdot \sqrt{\sum y_i^2})$$

Term	doc1	doc2	doc3	doc4	doc5	doc6
ship	1	0	1	0	0	0
boat	0	1	0	0	0	0
ocean	1	1	0	0	0	0
voyage	1	0	0	1	1	0
trip	0	0	0	1	0	1

WORKED CALCULATION: COS(DOC1, DOC2)
doc1=(1,0,1,1,0), doc2=(0,1,1,0,0). Dot product = 0+0+1+0+0 = 1. $|\text{doc1}| = \sqrt{3} \approx 1.732$, $|\text{doc2}| = \sqrt{2} \approx 1.414$.
 $\cos = 1 / (1.732 \times 1.414) \approx 0.408 \Rightarrow$ documents share the term "ocean" only, giving moderate similarity.

Applications: information retrieval (ranking search results), biological taxonomy, gene feature mapping.

2.6 Discrete Sequences — Edit Distance & LCSS

Edit Distance (Levenshtein Distance)

The **least total cost** to transform one sequence into another using a set of allowed operations (insertion, deletion, substitution/replacement), each with an assigned cost. The result depends critically on which operations are allowed and their costs.

WORKED EXAMPLE
Transform abababab ab into bababababa (a cyclic shift by one position):

- Using only **substitutions** ($a \leftrightarrow b$): needs **10 replacements** (every character differs).
- Using **insertion/deletion**: delete the leading "a", insert an "a" at the end $\rightarrow$ **1 deletion + 1 insertion** (much cheaper!).

This demonstrates that **the "right" edit distance depends entirely on the cost model chosen** for the application (e.g., DNA alignment penalizes indels differently from substitutions).

Longest Common Subsequence (LCSS)

Given two sequences, find the longest subsequence (not necessarily contiguous) common to both.

Sequences: xmjyauz and mzjawxu . LCSS = mjau (length 4) — m , then j , then a , then u appear in that relative order in both strings. LCSS can be computed in **polynomial time** via dynamic programming (unlike many other sequence problems).

2.7 Graph Similarity Measures

Task	Approach
Similarity between two nodes in the same graph, based on connectivity	Shortest paths (Dijkstra's algorithm) for a single path; PageRank / SimRank when many paths matter
Similarity between two whole graphs	Very challenging: the graph-isomorphism problem (deciding if two graphs are identical up to relabeling) is NP-hard . Practical measures: maximum common subgraph distance, substructure-based similarity, graph-edit distance.

2.8 Time Series — Dynamic Time Warping (DTW)

Naively applying an L_p -norm to two time series of equal length fails for several real-world reasons:

- Series are not on the same **scale** (behavioral attribute scaling/translation, e.g. amplitude differences)
- The absolute **time stamp** is not important — series may need to be shifted (temporal translation)
- Series contain **noisy segments**
- The **temporal axis scale** differs (one series is "stretched" relative to the other)

DYNAMIC TIME WARPING (DTW)

DTW **stretches the series along the time axis**, repeating some elements of either series, to produce two new sequences of equal length that are **aligned** to minimize distortion. Once aligned, any standard distance measure can be applied to the warped sequences. This is the standard technique for comparing time series that are "the same shape" but shifted/stretched in time (e.g. two people speaking the same word at different speeds).

2.9 High-Dimensional Data & the Curse of Dimensionality

Setting: $n \ll d$ (e.g. $n = 100$ observations in $d = 10^6$ dimensions) — common in genome analysis, spatial statistics, text/gene-expression data.

Curse of Dimensionality: Distances Concentrate as d Grows

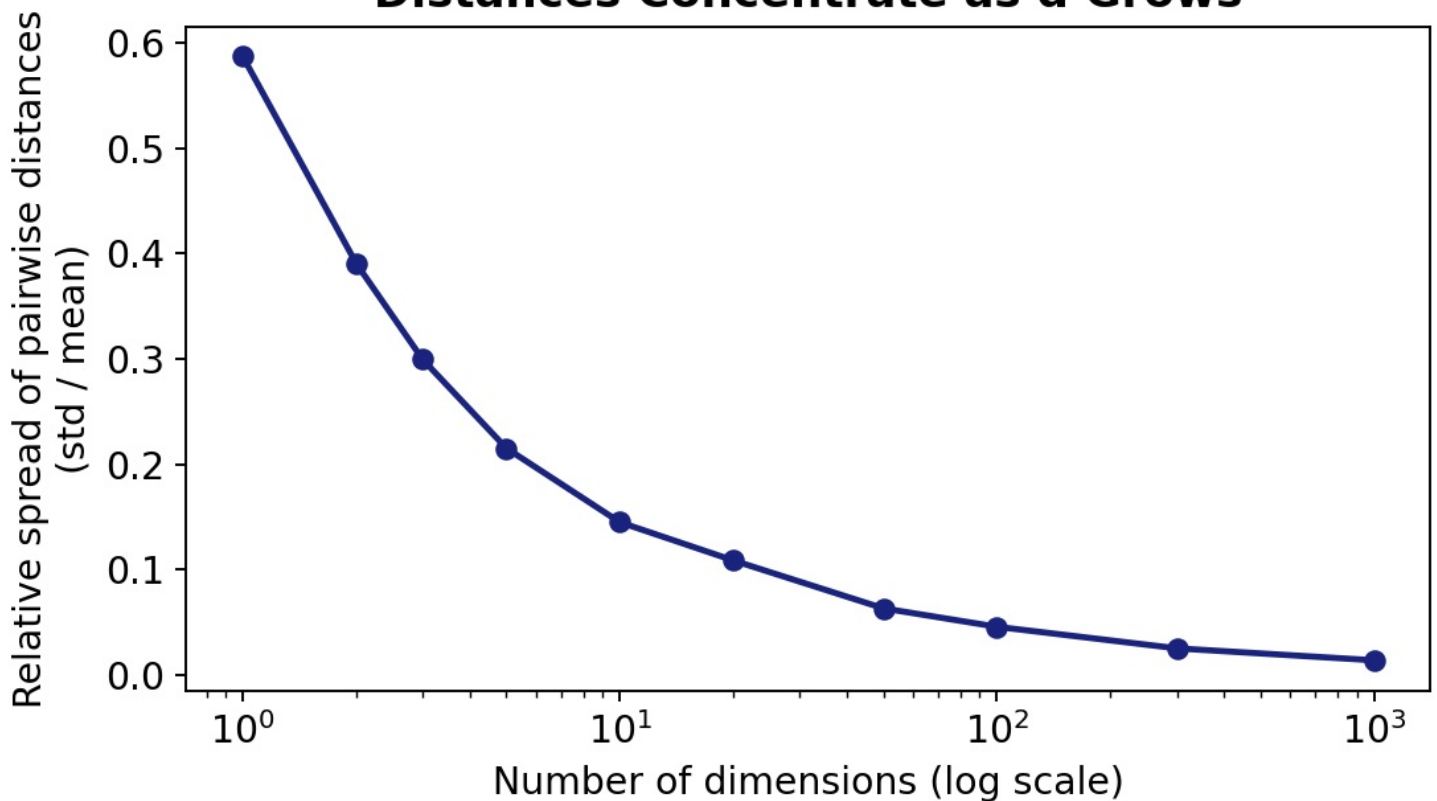


Fig. 2.2 — Empirical demonstration: as dimensionality d grows, the relative spread (std/mean) of pairwise distances collapses toward zero — i.e., **all points become almost equidistant**.

CONSEQUENCES OF THE CURSE OF DIMENSIONALITY

- The **center** of a high-dimensional space is almost empty; points scatter to corners and boundary regions.
- The **majority of random vectors become nearly orthogonal** to one another.
- **Almost all pairs of points end up at about the same distance**, so nearest-neighbor / distance-based notions (on which clustering and classification depend) lose discriminative power.
- Projecting onto a lower-dimensional subspace (e.g. PCA) may cause **severe information loss** if done carelessly.

This is why, per J. Leskovec, A. Rajaraman & J. Ullman ("Mining of Massive Datasets"), **clustering in 2D "looks easy" but this intuition is deeply misleading** once real applications with 10–10,000+ dimensions are involved.

2.10 Part II Question & Answer Bank

Q2.1 Prove/explain why $L^\infty(x,y) \leq L^2(x,y) \leq L^1(x,y)$ always holds for any two points in $\mathbb{R}^d$.

Answer: Let $a_i = |x_i - y_i| \geq 0$. $L^\infty = \max_i a_i$. Since $(\max_i a_i)^2 \leq \sum a_i^2 \leq (\sum a_i)^2$ (the last inequality holds because all terms are non-negative, so squaring the sum only adds non-negative cross-terms), taking square roots gives $\max_i a_i \leq \sqrt{\sum a_i^2} \leq \sum a_i$, i.e. $L^\infty \leq L^2 \leq L^1$. Equality throughout holds only if at most one a_i is non-zero.

Q2.2 Why is cosine similarity preferred over Euclidean distance for comparing text documents of very different lengths?

Answer: Euclidean distance grows with the magnitude of the vectors — a long document that repeats the same words proportionally as a short document would appear "far" from it in Euclidean space purely due to scale, even though their topical content (direction) is identical. Cosine similarity normalizes by vector length (dividing by the product of norms), measuring only the **angle** between vectors, so it correctly identifies the two documents as similar regardless of length.

Q2.3 Two DNA-like sequences differ only by a single-character insertion near the start (causing everything after that point to be "shifted" by one position). Compare what a naive Hamming/substitution-only edit distance would report vs. an edit distance that also allows insertions/deletions, and explain the implication for choosing a distance function.

Answer: A substitution-only metric would see almost every subsequent character as mismatched (since positions are shifted by 1), reporting a huge distance despite the sequences being nearly identical structurally. An edit distance allowing indels would correctly find a distance of 1 (a single insertion). **Implication:** the chosen cost model must reflect the actual generative process of differences in the domain (this is precisely why DNA alignment tools like Needleman-Wunsch use gap penalties rather than pure substitution costs).

Q2.4 Why can't the graph-isomorphism-based "identical graphs" test be used directly as a practical similarity measure for large graphs?

Answer: Determining whether two graphs are isomorphic (an exact match up to node relabeling) is NP-hard in general (no known polynomial-time algorithm, though it is not proven NP-complete either — it occupies a special complexity class). For large graphs this is computationally infeasible, so practitioners fall back on tractable approximate similarity measures: maximum common subgraph, substructure/fingerprint-based similarity, or graph-edit distance with bounded search.

Q2.5 [Research-oriented] Explain, mechanistically, why Euclidean nearest-neighbor search becomes meaningless as dimensionality grows (the curse of dimensionality), and name one mitigation strategy.

Answer: For i.i.d. coordinates, by the law of large numbers the squared Euclidean distance is (approximately) a sum of d i.i.d. terms; its **mean grows linearly in d** while its **standard deviation grows only as $\sqrt{d}$** , so the coefficient of variation (std/mean) shrinks as $1/\sqrt{d} \rightarrow 0$. Consequently the ratio of the farthest to nearest neighbor distance tends to 1, i.e. every point looks roughly equidistant from a query point — nearest-neighbor structure vanishes. **Mitigations:** dimensionality reduction (PCA, feature selection), using fractional- p Minkowski distances ($p < 1$, "fractional norms") which are empirically less affected, or switching to subspace/projected clustering that only considers a relevant subset of dimensions at a time.

Q2.6 Given points $x_1=(1,2)$, $x_2=(3,5)$ compute the Manhattan, Euclidean, and Maximum distance between them.

Answer: $|\Delta_1|=2$, $|\Delta_2|=3$. Manhattan = $2+3 = 5$. Euclidean = $\sqrt{4+9} = \sqrt{13} \approx 3.61$. Maximum = $\max(2,3) = 3$. (Matches the reference distance matrices in §2.3.)

Clustering

3.1 Definitions, Applications & Domains

Cluster: a collection of data objects that are (a) similar/related to one another *within* the same group and (b) dissimilar/unrelated to objects in *other* groups.

Clustering (a.k.a. cluster analysis, data segmentation): finding similarities between data according to characteristics found in the data, then grouping similar objects into clusters — **without pre-existing labels**.

Two Roles of Clustering

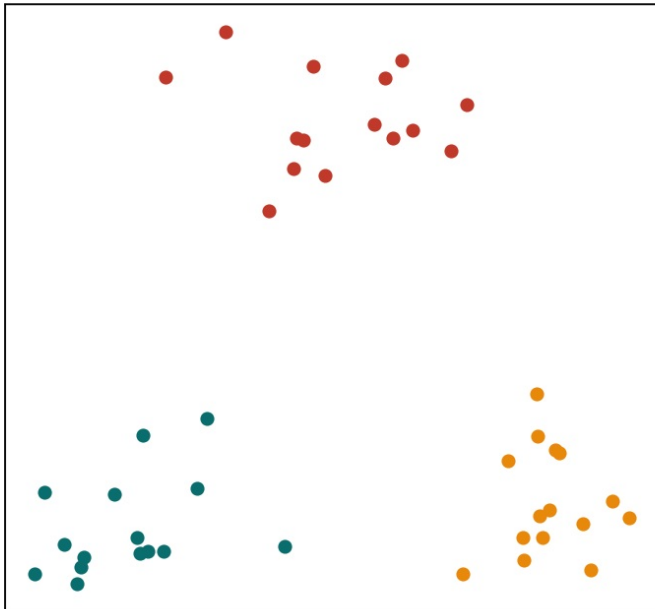
- **Stand-alone tool** to get insight into a data distribution (exploratory data analysis)
- **Preprocessing step** for other algorithms (e.g., outlier/anomaly detection — points that do not fit well into any cluster are candidate outliers)

Application Domains

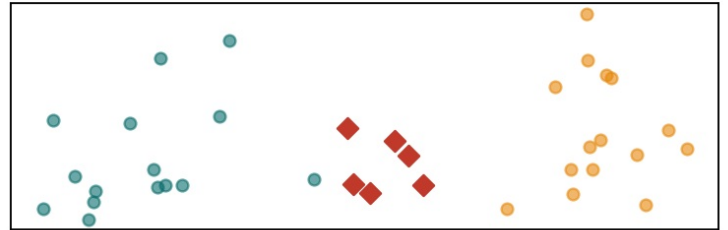
Domain	Use of clustering
Biology	Taxonomy of living things: kingdom, phylum, class, order, family, genus, species
Information Retrieval	Document clustering
Land Use	Identifying areas of similar land use in earth-observation databases
Marketing	Discovering distinct customer segments to design targeted campaigns
City Planning	Grouping houses by type, value, and geographic location
Climate Science	Understanding earth climate; finding atmospheric/ocean patterns
Economics	Market research

3.2 Design Considerations for Clustering

Exclusive clustering (each point -> 1 cluster)



Overlapping clustering (points can belong to >1 cluster)



Distance-based (compact, convex regions)



Density-based (arbitrary shapes)

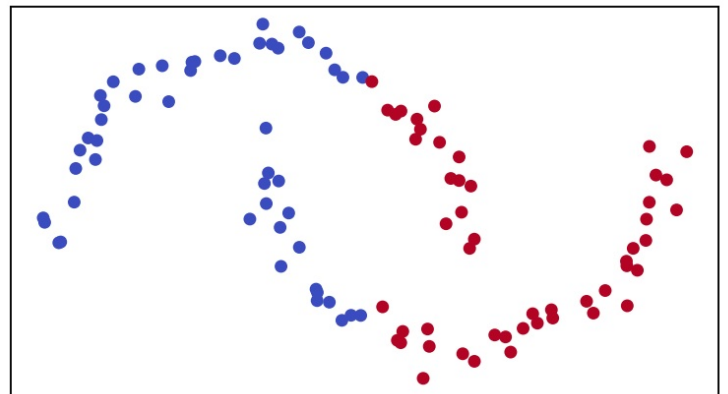


Fig. 3.1 — Four independent design axes for any clustering method: exclusive vs. overlapping cluster membership (top), distance-based vs. density-based similarity notion (bottom).

Design axis	Options
Partitioning criteria	Single-level vs. hierarchical (often multi-level hierarchical partitioning is desirable, giving a full dendrogram rather than one flat cut)
Separation of clusters	Exclusive (each object belongs to exactly one cluster, e.g. one customer → one region) vs. non-exclusive/overlapping (one object may belong to >1 cluster, e.g. one document may span multiple topic classes)
Similarity measure	Distance-based (Euclidean, road-network, vector similarity — compact convex clusters) vs. connectivity-based (density or contiguity — arbitrarily shaped clusters)
Clustering space	Full space (typical for low-dimensional data) vs. subspaces (typical for high-dimensional data, mitigating the curse of dimensionality by considering only relevant dimension subsets)

3.3 Cluster Quality: Preconditions & Goals

PRECONDITION OFTEN OVERLOOKED

Clustering presumes that **existing patterns actually exist within the data**. Before clustering, ask: does the application give hints for such a hypothesis? If the data is truly uniformly/randomly distributed, "clusters" produced by an algorithm are meaningless artifacts. (This motivates the Hopkins statistic in §3.9.)

A **good clustering** method produces clusters with:

- **High intra-class similarity:** objects within a cluster are cohesive

- **Low inter-class similarity:** objects in different clusters are distinctive

The overall **quality** of a clustering method depends on: (1) the similarity measure it uses, (2) its implementation, and (3) its ability to discover some or all of the hidden patterns actually present in the data.

WHY CLUSTERING IS A GENUINELY HARD PROBLEM

Clustering in 2 dimensions with small data "looks easy," and in most such cases looks are not deceiving. But real applications routinely involve 10 to 10,000+ dimensions, and (per §2.9) high-dimensional spaces behave very differently: almost all pairs of points sit at about the same distance, and almost any two vectors are nearly orthogonal — the "Curse of Dimensionality" (Leskovec, Rajaraman & Ullman, *Mining of Massive Datasets*).

3.4 Overview of Major Clustering Approaches

Family	Core idea	Typical methods
Partitioning	Construct various partitions, evaluate by a criterion (e.g. minimize sum of squared errors)	k-Means, k-Medoids, CLARANS
Hierarchical	Build a hierarchical decomposition of the data using merge/split criteria	AGNES, DIANA, BIRCH, CAMELEON
Density-Based	Clusters = dense regions separated by sparse (noise) regions; based on connectivity & density functions	DBSCAN, OPTICS, DenClue
Grid-Based	Quantize space into a multi-level granularity grid structure	STING, WaveCluster, CLIQUE

3.5 Partitioning Methods: k-Means

Formal setting: a set $X = \{x_1, \dots, x_n\}$ of n objects is partitioned into k clusters C_i , $i=1..k$. Distance is measured to the **centroid** (or medoid) c_i of cluster C_i :

$$\text{centroid} = (1/n) \sum_{i=1}^n x_i \quad \text{medoid} = \operatorname{argmin}_{y \in X} \sum_{i=1}^n \text{dist}(y, x_i)$$

Objective: minimize the sum of squared distances (within-cluster variance):

$$E = \sum_{i=1}^k \sum_{p \in C_i} \text{dist}(p, c_i)^2$$

The Lloyd / MacQueen k-Means Algorithm

1. Partition objects into k nonempty subsets (initial, often random, assignment)
 2. Repeat:
 - a. Compute centroid (mean point) for each partition
 - b. Assign each object to the cluster of its nearest centroid
- Until: no change (convergence)

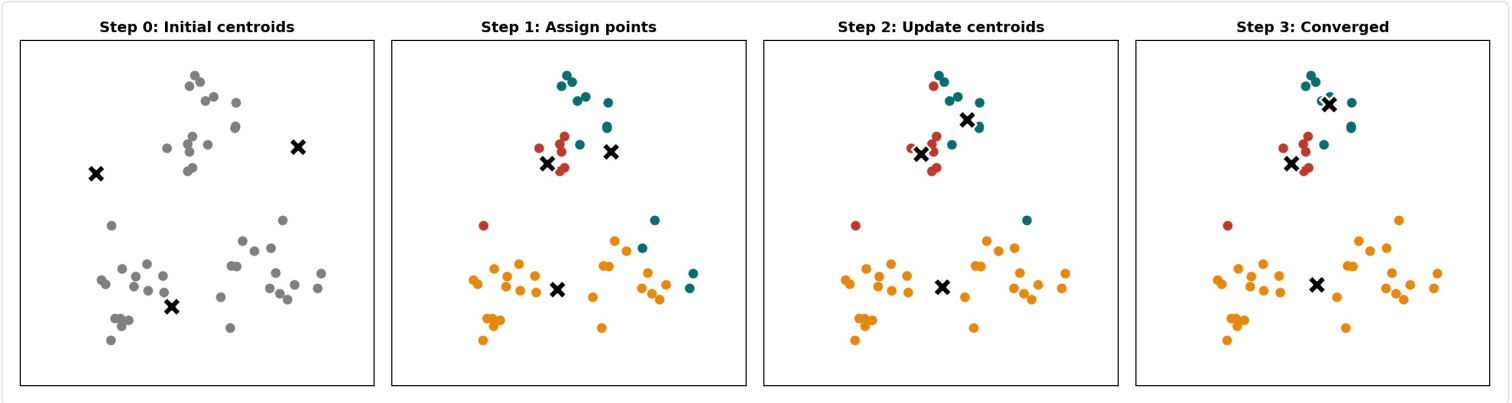


Fig. 3.2 — The k-Means iteration in action (k=3): random initial centroids → assignment → centroid update → converged result.

GLOBAL VS. HEURISTIC OPTIMIZATION

Global optimum would require exhaustively enumerating all possible partitions of n points into k clusters — combinatorially infeasible (this is exactly why k-Means's underlying optimization problem is **NP-hard**). k-Means (Lloyd 1957/1982, MacQueen 1967) is a **heuristic**: it is only guaranteed to converge to a *local* optimum, and the final result depends on the random initialization. k-Medoids/PAM (Kaufman & Rousseeuw 1987) is an alternative heuristic representing each cluster by an actual data point rather than a computed mean.

3.6 Partitioning Methods: k-Medoids / PAM

THE K-MEANS OUTLIER PROBLEM

k-Means is **sensitive to outliers**: because the centroid is a computed *mean*, a single object with an extreme value can substantially distort a cluster's representative point, dragging it away from the "true" bulk of the cluster.

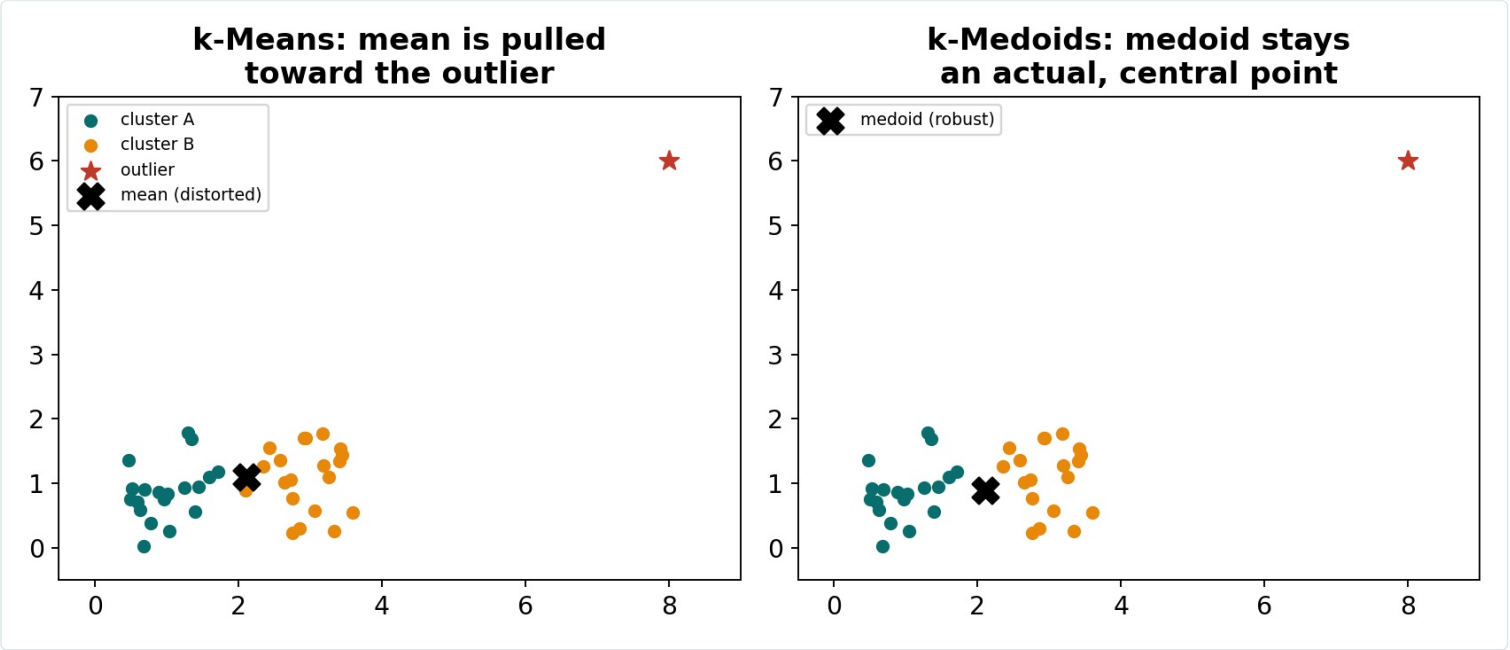


Fig. 3.3 — Left: an outlier drags the k-Means mean away from the true cluster centers. Right: k-Medoids' medoid remains an actual, centrally-located data point, robust to the outlier.

k-Medoids (PAM — Partition Around Medoids): instead of taking the mean value of objects in a cluster as the reference point, use a **medoid** — the most centrally located *actual object* in the cluster (i.e., the object minimizing the sum of distances to all other objects in its cluster). Because a medoid must be a real data point, extreme outlier values cannot become the reference point themselves, making PAM substantially more robust — at the cost of higher computational complexity per iteration (must evaluate real candidate points, not just compute an arithmetic mean).

3.7 Hierarchical Clustering

Main idea: define a distance between clusters (not just between points), then merge/split clusters based on that distance, using the resulting distance matrix as the sole clustering criterion.

TWO DEFINING CHARACTERISTICS

- Hierarchical clustering **does not require the number of clusters k as an input** — the whole hierarchy (dendrogram) is built.
- Hierarchical clustering **needs a termination condition** to decide where to "cut" the hierarchy into a usable flat partition.

Distances Between Clusters (Linkage Criteria)

Linkage	Formula	Behavior
Single	$d_{SL}(G,H) = \min_{x \in G, y \in H} d(x,y)$	Distance of the two <i>closest</i> members across clusters. Can produce long, "chained" clusters (chaining effect).
Complete	$d_{CL}(G,H) = \max_{x \in G, y \in H} d(x,y)$	Distance of the two <i>most distant</i> members. Produces compact, roughly equal-diameter clusters.
Average	$d_{Avg}(G,H) = (1/(n_G n_H)) \sum_{x \in G} \sum_{y \in H} d(x,y)$	Mean pairwise distance across all cross-cluster pairs. A compromise between single and complete linkage.

AGNES (Agglomerative Nesting) vs. DIANA (Divisive Analysis)

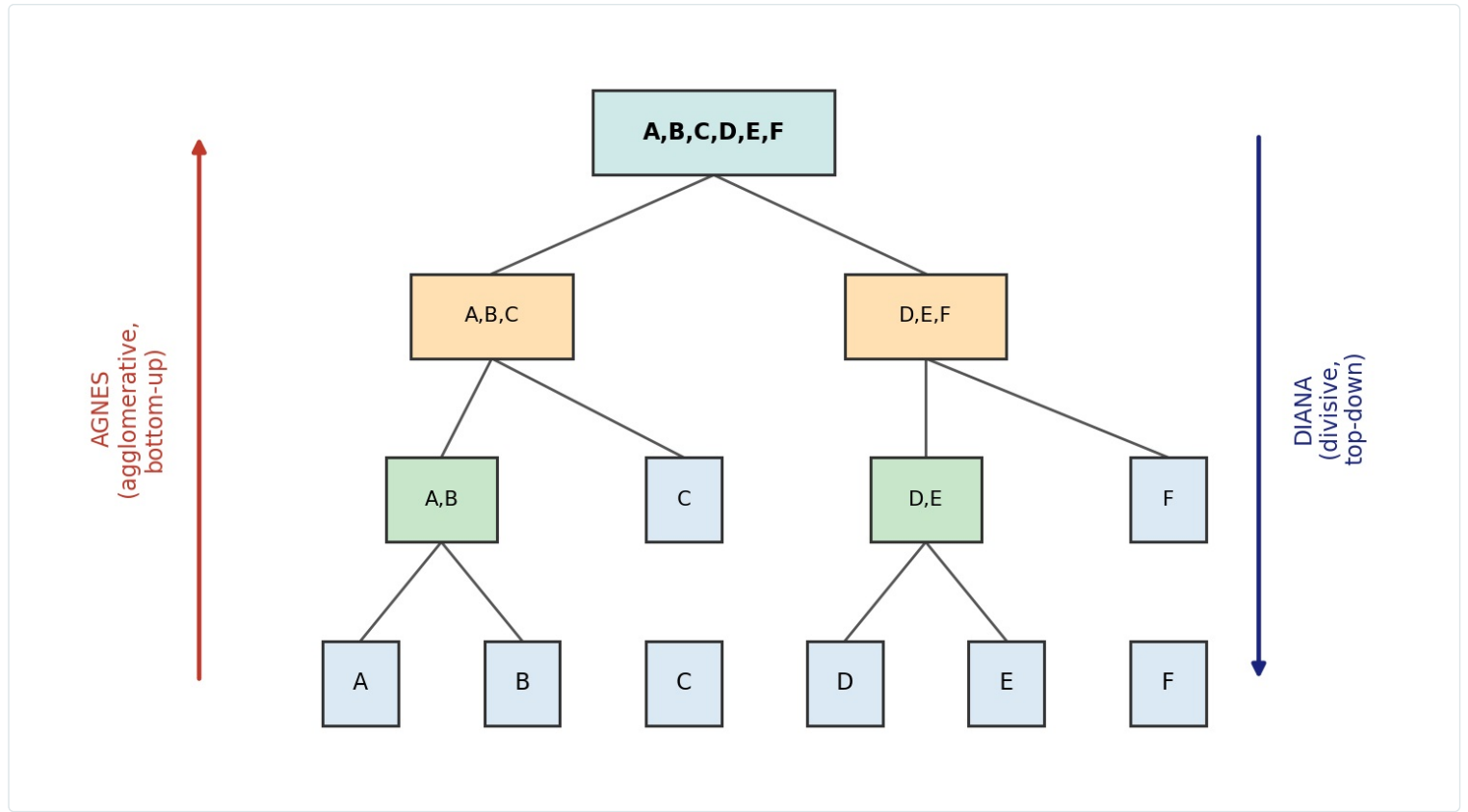


Fig. 3.4 — AGNES builds the dendrogram bottom-up (each point starts as its own cluster, most-similar pairs merge iteratively). DIANA is the mirror-image top-down process.

	AGNES (Agglomerative)	DIANA (Divisive)
Direction	Bottom-up	Top-down
Start state	Every point is its own cluster	All points form a single cluster
End state	All points eventually merge into one cluster	Every point eventually forms its own cluster
Core mechanism	Repeatedly merge the pair of clusters with least dissimilarity (uses the dissimilarity matrix directly, classically with single-link)	Repeatedly split a cluster into two, choosing the split that most increases separation
Reference	Kaufman, L., Rousseeuw, P. (1990). <i>Finding Groups in Data: An Introduction to Cluster Analysis</i> .	

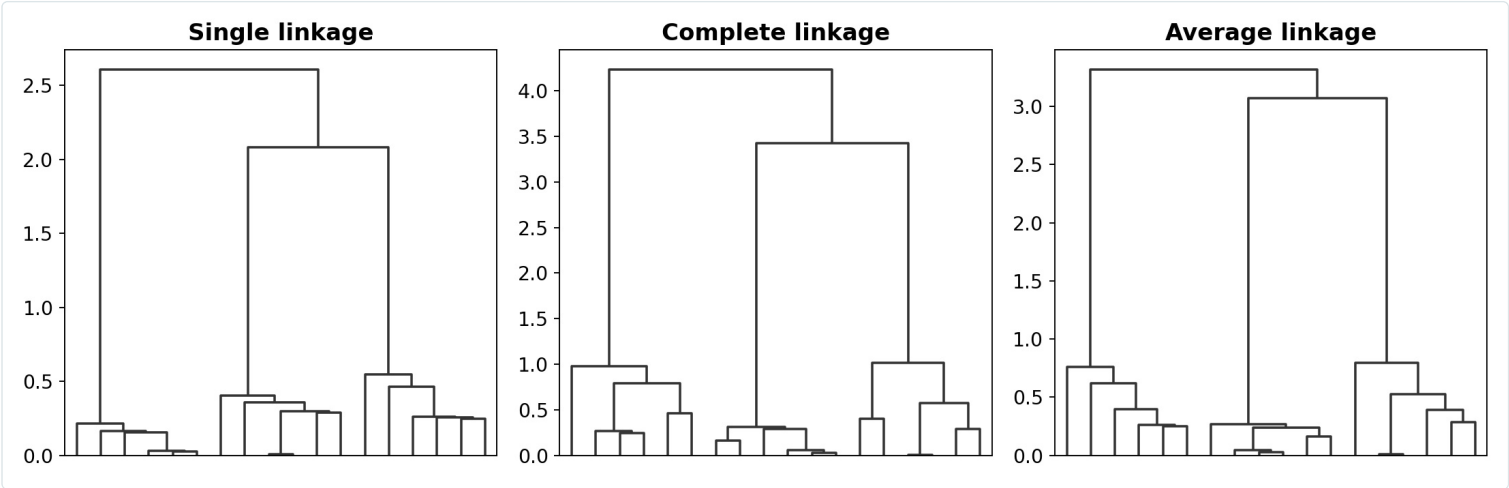


Fig. 3.5 — The same synthetic 3-group dataset clustered with single, complete, and average linkage. Single linkage is visibly prone to chaining (uneven, "staircase" merges); complete linkage yields the most balanced dendrogram.

FREQUENTLY-TESTED FACT: LINKAGE CHOICE CHANGES THE RESULT

On the Iris dataset, single-linkage, complete-linkage, and average-linkage Euclidean dendrograms produce **visibly different** cluster groupings and different apparent "natural" cut points — even though all three start from the *same* raw distance matrix. This directly illustrates §2.1's Aggarwal quote: the choice of (dis)similarity/linkage function is not a minor implementation detail, it materially changes the discovered structure.

Pros & Cons of Hierarchical Clustering

- Pros
- Cons

- Flexibility from using arbitrary/complex distance measures
- No parameters beyond the distance function and fusion (linkage) method — notably, **no need to fix k in advance**
- Result exposes substructures at multiple resolutions (full hierarchy)
- Significant effort to analyze the result (a full dendrogram, not one flat answer)
- Provides *many* partitions of the data (one per possible cut) — no single canonical clustering
- The user must still decide how to partition (where to cut)

COMPLEXITY

General hierarchical clustering: $O(N^3)$ time (N = number of data points), which can be reduced to $O(N^2 \log N)$ using efficient data structures for maintaining the nearest-cluster information. The **single-link** case specifically can be computed in $O(N^2)$ time (e.g., via the SLINK algorithm).

3.8 Density-Based Clustering: DBSCAN

DBSCAN = **D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise (Ester, Kriegel, Sander & Xu, KDD 1996).

Parameters

- ϵ (**eps**): the maximum distance within which two points are considered "reachable" from each other
- **minPts**: the minimum number of points (including the point itself) required within an ϵ -neighborhood to call a point a **core point**

Types of Points

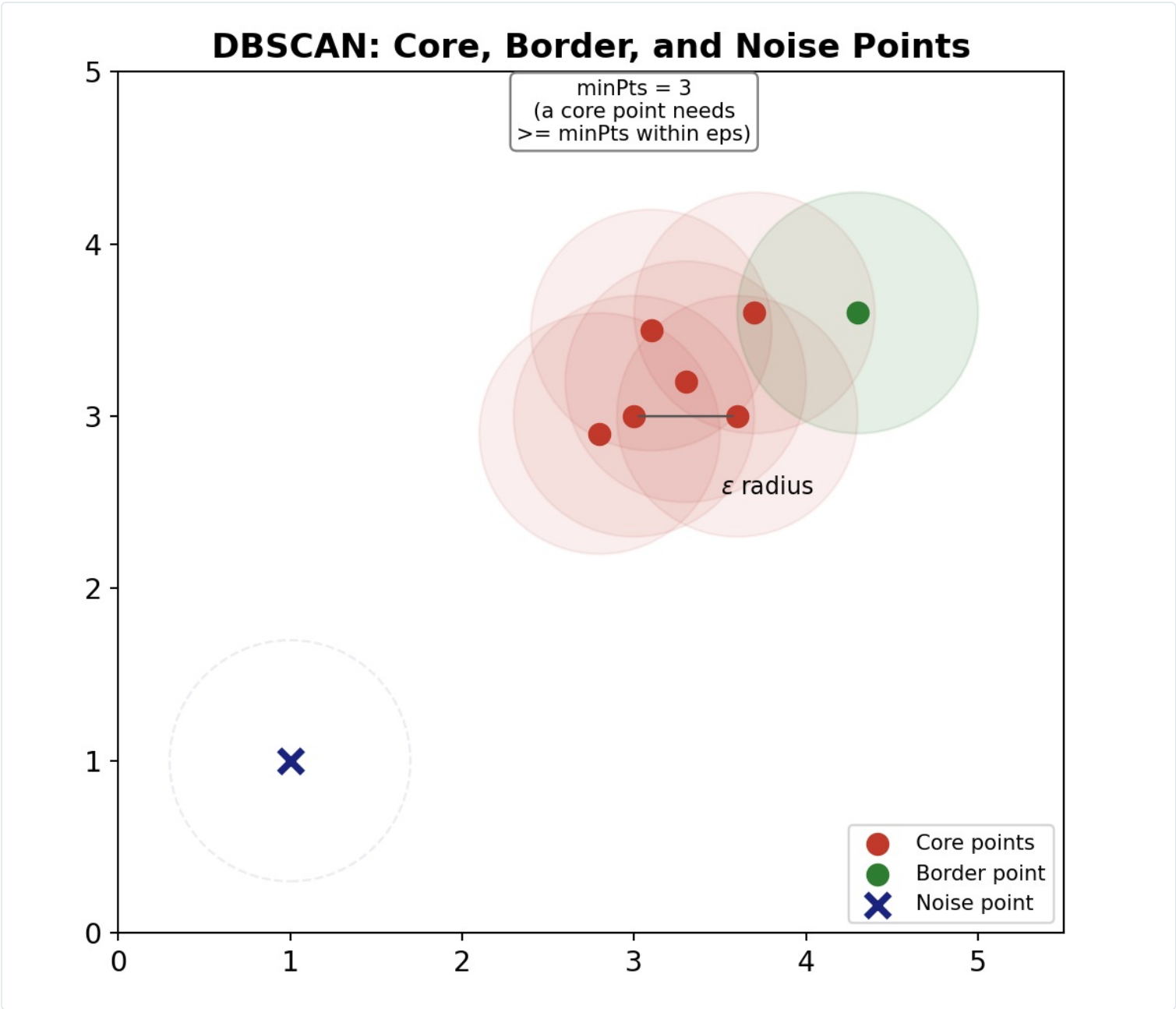


Fig. 3.6 — Core, border, and noise points under DBSCAN with minPts = 3.

Type	Definition
------	------------

Core	At least minPts points (including itself) within distance ϵ
Border	Connected to (within ϵ of) a core point, but itself has fewer than minPts points within ϵ
Noise	Neither core nor border — does not belong to any cluster

DBSCAN Pseudocode

```
DBSCAN(D, eps, MinPts)
  C = 0
  for each unvisited point P in dataset D
    mark P as visited
    NeighborPts = D.regionQuery(P, eps)
    if sizeof(NeighborPts) < MinPts
      mark P as NOISE
    else
      C = next cluster
      expandCluster(P, NeighborPts, C, eps, MinPts)

expandCluster(P, NeighborPts, C, eps, MinPts)
  add P to cluster C
  for each point P' in NeighborPts
    if P' is not visited
      mark P' as visited
      NeighborPts' = D.regionQuery(P', eps)
      if sizeof(NeighborPts') >= MinPts
        NeighborPts = NeighborPts joined with NeighborPts'
    if P' is not yet member of any cluster
      add P' to cluster C
      unmark P' as NOISE if necessary

regionQuery(P, eps)
  return all points within P's eps-neighborhood (including P)
```

DBSCAN vs. k-Means — the Critical Comparison

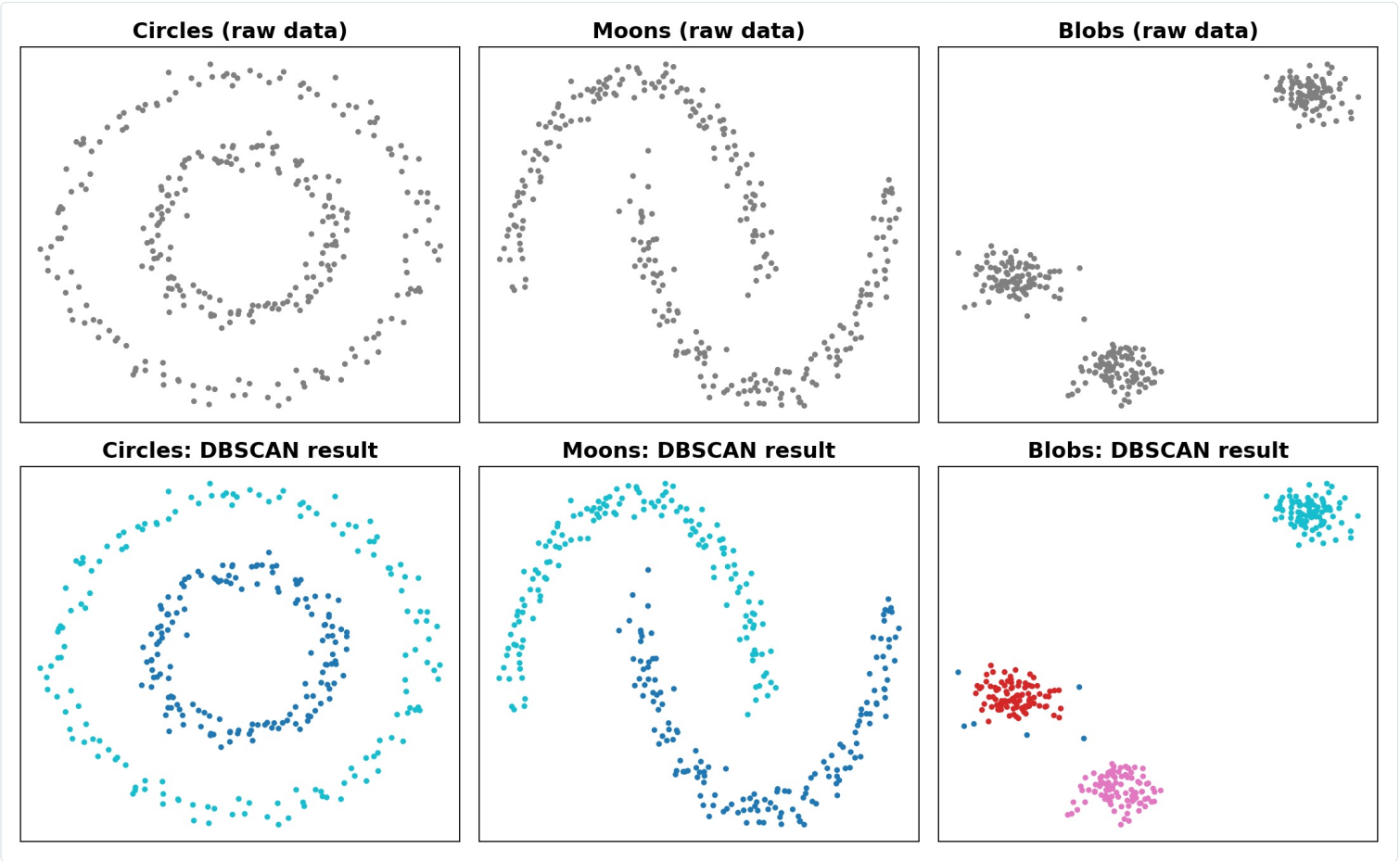


Fig. 3.7 — DBSCAN correctly separates non-convex "circles" and "moons" datasets by density; it also isolates a genuinely different-density third group as its own cluster in the "blobs" case.

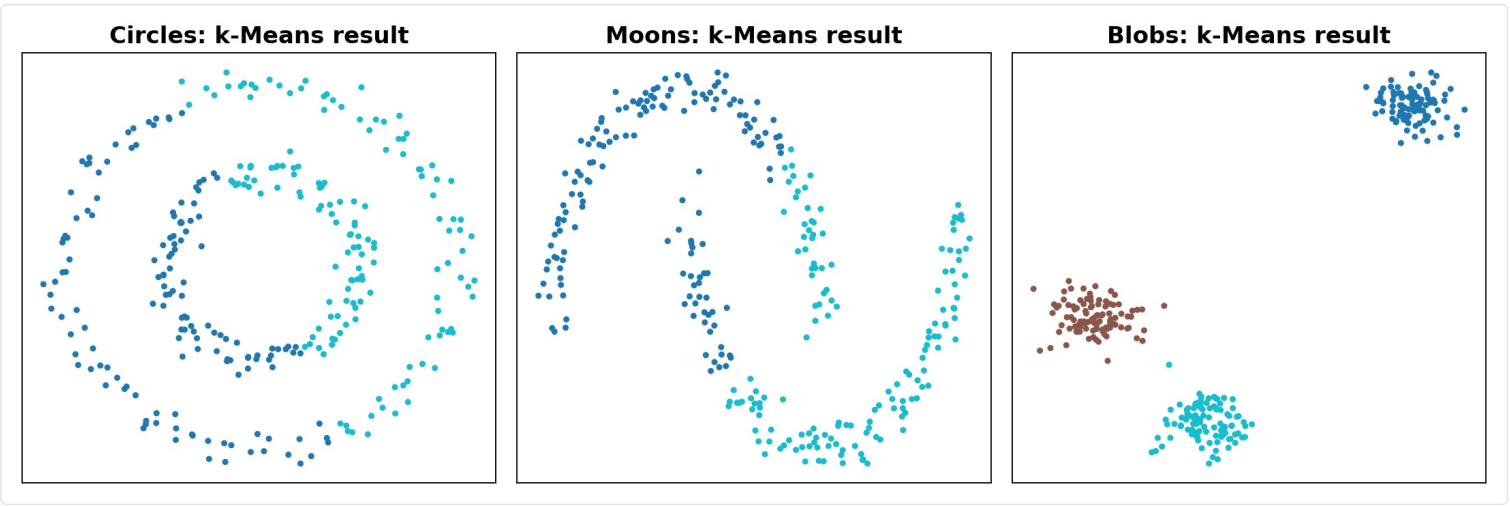


Fig. 3.8 — *k*-Means applied to the same datasets fails on circles/moons: it always produces convex (Voronoi-cell-shaped) partitions, cutting straight through the natural ring/crescent structure because it has no notion of connectivity or density.

Criterion	k-Means	DBSCAN
Cluster shape	Convex only (Voronoi cells around centroids)	Arbitrary shape (follows density connectivity)
Number of clusters	Must be specified in advance (<i>k</i>)	Determined automatically from data + (ϵ , minPts)
Outliers / noise	Every point is forced into some cluster	Explicitly labels noise points; does not force-fit them
Sensitivity	Sensitive to initialization & outliers (via the mean)	Sensitive to the choice of ϵ and minPts; struggles with varying-density clusters
Underlying assumption	Clusters are compact, similarly-sized, globular	Clusters are dense regions separated by sparser regions

3.9 Cluster Assessment

(a) Assessing Clustering Tendency — Should We Even Cluster?

Before trusting any clustering result, first assess whether a **non-random structure** exists at all, i.e. estimate the probability that the data was in fact generated by a uniform/random distribution. The standard statistical test is the **Hopkins Statistic** (Cross & Jain, 1982), which compares nearest-neighbor distances of real data points against distances expected under uniform-random sampling.

A visual heuristic (Ordered Dissimilarity/ODI plot): reorder a dissimilarity matrix by clustering and display it as a heatmap. Real, clusterable data (e.g. Iris) shows clear **block-diagonal structure** (dark blocks along the diagonal); purely random data shows no such blocks — visually confirming (or refuting) clustering tendency before committing to an algorithm.

(b) Determining the Number of Clusters *k*

Elbow Method

Elbow Method for Optimal k

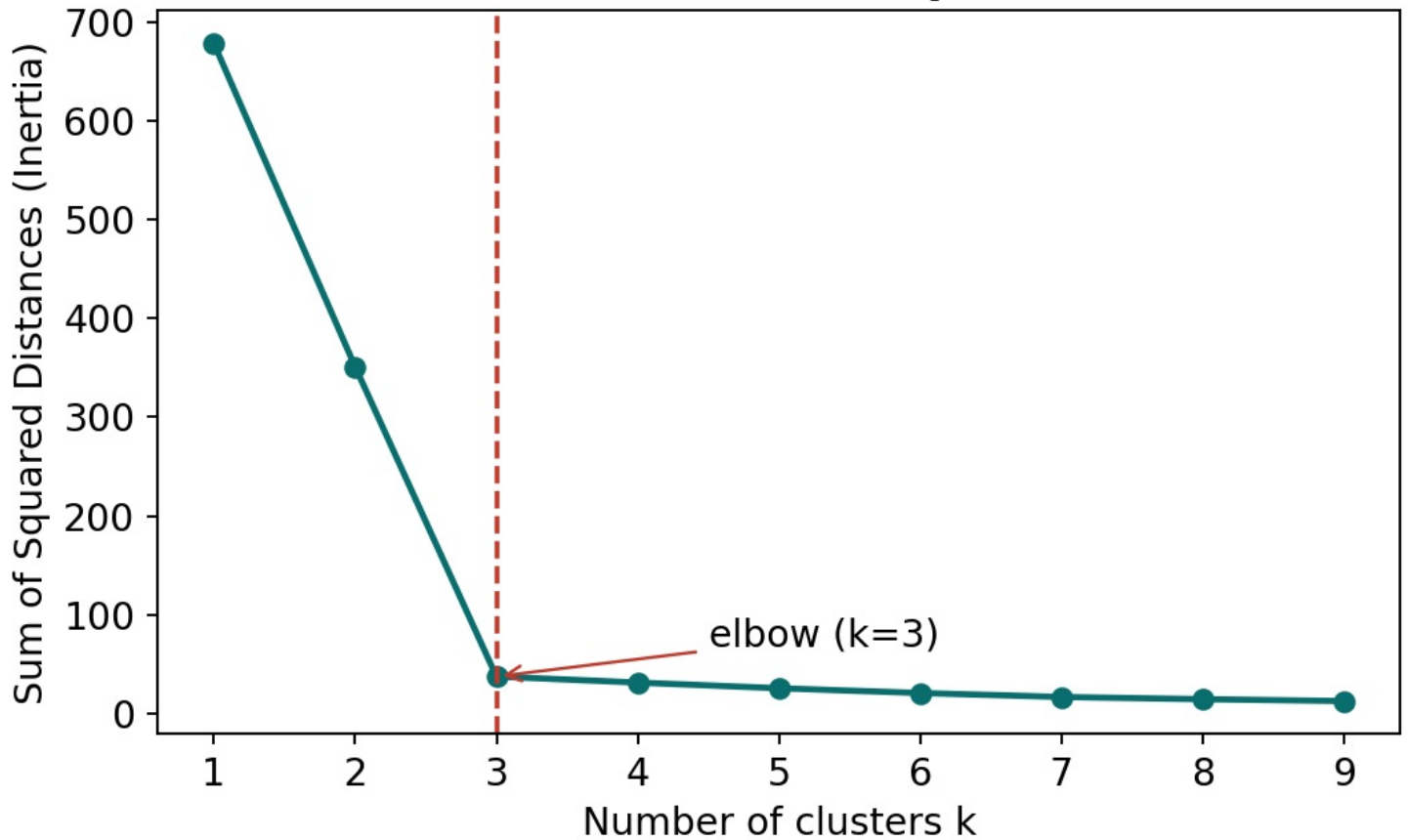


Fig. 3.9 — Total within-cluster sum of squares (inertia) vs. k . The "elbow" (sharp bend, here at $k=3$) is taken as the optimal number of clusters.

Compute the sum of squared distances (E , §3.5) for each candidate k ; plot it against k . As k increases, E monotonically decreases (more clusters can only reduce within-cluster variance) — the optimal k is chosen at the point of diminishing returns, the "elbow." **Weakness:** highly subjective — real elbows are often smooth curves with no unambiguous bend.

Silhouette Method

Silhouette Method for Optimal k

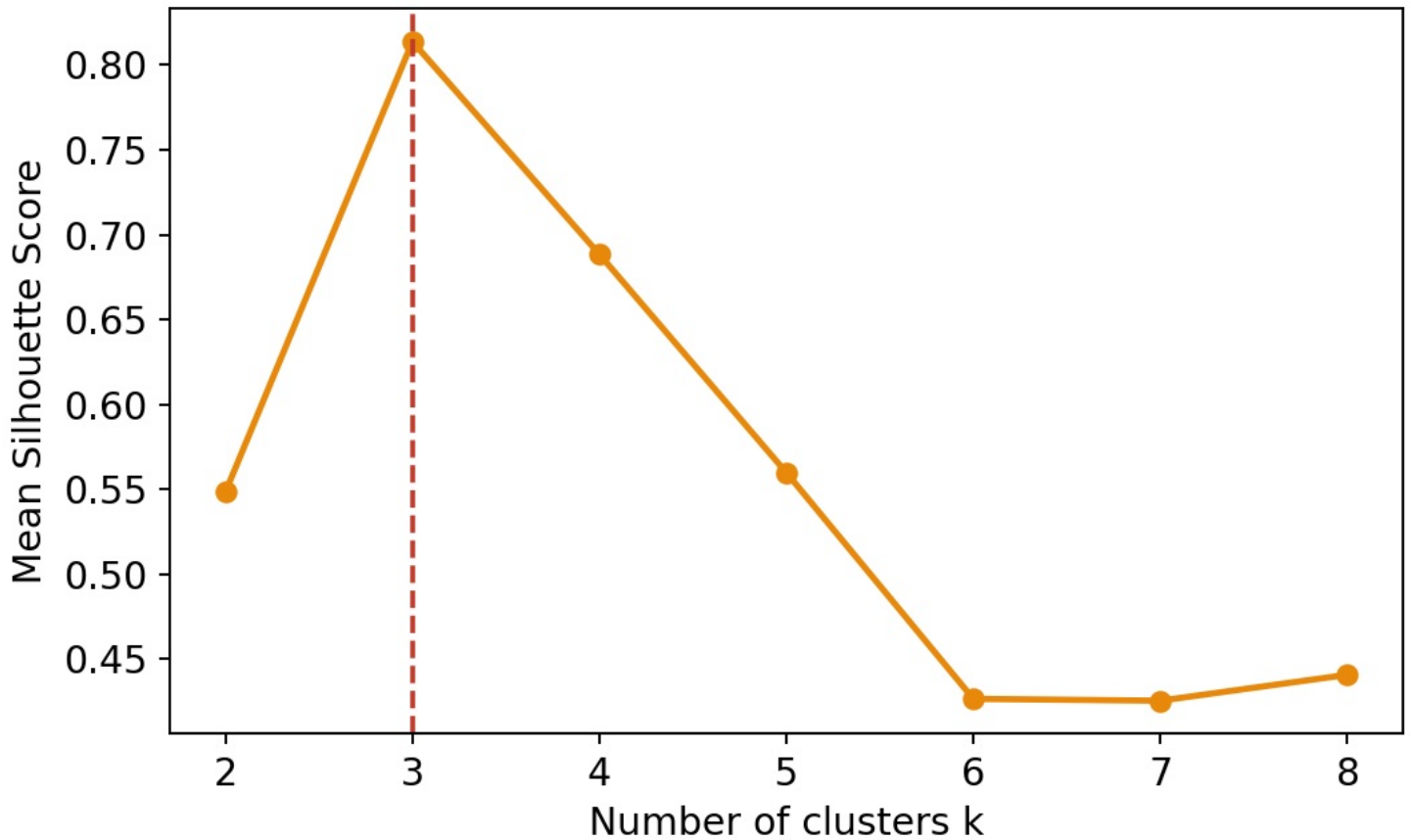


Fig. 3.10 — Mean silhouette score vs. k. The peak indicates the k that best balances compactness and separation.

$$sc(i) = (b_i - a_i) / \max(a_i, b_i) \quad -1 \leq sc(i) \leq 1$$

- **a_i** : mean distance from point i to all other points in the *same* cluster (measure of compactness — lower is better)
- **b_i** : mean distance from point i to all points in the *nearest other* cluster (measure of separation — higher is better)

The **silhouette score of a clustering K** is the mean silhouette coefficient over all instances. Values near +1 indicate well-separated, compact clusters; values near 0 indicate overlapping clusters; negative values suggest points may be assigned to the wrong cluster.

(c) External Evaluation (requires ground truth)

Metric	Formula / idea
Purity	For each cluster, take the fraction of the majority (true) class; average, weighted by cluster size (Murphy, <i>Probabilistic Machine Learning</i> , 2022, p.712)
Jaccard Index	$J(A,B) = A \cap B / A \cup B $ — overlap between predicted cluster A and true class B
F-score / Confusion Matrix	Precision/recall style metrics adapted from the supervised-learning lecture, treating each true class as if it were the "positive" label to predict

(d) Manual and Indirect Evaluation

- **Manual evaluation**: a domain expert visually/qualitatively assesses whether clusters "make sense"
- **Indirect evaluation**: assess the clustering by how well it improves a downstream application (e.g., does clustering-based preprocessing improve a recommender system's accuracy?)

3.10 Aspects of Big-Data Clustering

CORE CHALLENGE: SCALABILITY

Clustering algorithms often carry **high time complexity** — e.g., the k-Means optimization problem is **NP-hard** in general, and naive hierarchical clustering is $O(N^3)$. The engineering goal is to **speed up / scale up** these algorithms with **minimum sacrifice to clustering quality**.

Typical clustering procedure: (1) **Initialization** (e.g., randomly choose k cluster centers); (2) an **iterative process** until some

convergence criterion (e.g., in k-Means, assign each point to its closest cluster center, then recompute centers).

Speed-Up / Scale-Up Strategies

- **Reducing the iteration number** — smarter initialization (e.g. k-Means++) so fewer refinement rounds are needed
- **Reducing access to data points** — randomized/sampling techniques (cluster on a representative sample, not the full dataset)
- **Distributing/parallelizing computation** — e.g. MapReduce-style k-Means, where the assignment step is embarrassingly parallel across data partitions

(Aggarwal, C., Reddy, K. (eds.), *Data Clustering: Algorithms and Applications*, Chapman & Hall/CRC, 2014.)

3.11 Part III Question & Answer Bank

Q3.1 Derive/explain why the k-Means objective function E is guaranteed to (weakly) decrease at every iteration, yet the algorithm is not guaranteed to find the global optimum.

Answer: Each Lloyd iteration has two steps, both of which cannot increase E: (1) the **assignment step** reassigns every point to its *nearest* centroid, which by definition cannot increase that point's squared distance term; (2) the **update step** recomputes each cluster's centroid as the arithmetic mean, which is exactly the point that *minimizes* the sum of squared distances to all points currently assigned to that cluster. Since E is bounded below by 0 and strictly non-increasing, the algorithm converges to a fixed point — but this fixed point is only a **local** minimum of the (NP-hard, non-convex) global objective; different random initializations of the k centroids can converge to different local optima, which is why practitioners run k-Means multiple times with different seeds (e.g. k-Means++, or $n_{\text{init}} > 1$) and keep the best result.

Q3.2 A dataset contains two concentric rings (an inner circle cluster and an outer ring cluster). Explain in detail why k-Means with k=2 will fail on this data, and why DBSCAN succeeds. What ϵ and minPts considerations arise?

Answer: k-Means partitions space into Voronoi cells around two centroids — this always yields **convex** regions (each cell is an intersection of half-planes). Two concentric rings are not convex-separable: any straight decision boundary that best separates the two centroids will slice through both rings rather than isolating one ring from the other, because the true clusters wrap around each other in a non-convex way. DBSCAN instead relies on **density-connectivity**: as long as points along each ring are locally dense enough (within ϵ of minPts neighbors), a chain of nearby points connects the whole ring into a single cluster regardless of its global (non-convex) shape, while the "empty" annular gap between the rings — with no points at all — naturally prevents the two rings from being connected to each other. **Practical caveat:** ϵ must be small enough that the gap between the rings is larger than ϵ (so DBSCAN does not merge them), yet large enough that consecutive points along a ring are still mutually reachable — DBSCAN can fail if the two rings have very different local densities, since a single global ϵ may not fit both.

Q3.3 Compare single-linkage and complete-linkage AGNES in terms of the "chaining effect." Which is more robust to a thin bridge of noise points connecting two otherwise well-separated clusters?

Answer: Single linkage merges clusters based on their *single closest pair* of points, so a thin "bridge" of a few noise points positioned between two dense clusters can cause single linkage to chain them together step by step (each bridge point is close to the next), ultimately merging two clusters that a human would consider distinct — this is the classic **chaining effect**. Complete linkage instead uses the *most distant* pair, meaning two clusters are only merged once *even their worst-case (farthest) points* are close — a thin bridge does not help, because the bulk of each cluster remains far from the bulk of the other. Complete linkage is therefore substantially more robust to noise bridges, at the cost of being biased toward finding compact, similarly-sized clusters and potentially splitting genuinely elongated clusters.

Q3.4 Explain the difference between "purity" as an external cluster-validation metric and "silhouette score" as an internal one — and describe a scenario where a clustering could score high on one but low on the other.

Answer: Purity requires **ground-truth class labels** and measures how well clusters align with those known classes (the fraction of each cluster occupied by its majority true class). Silhouette score requires **no external labels** — it purely measures internal geometric quality (intra-cluster compactness vs. inter-cluster separation) using the distance function alone. **Divergent scenario:** if the true classes are *not* geometrically well-separated in feature space (e.g., two biological species that overlap heavily in the chosen measurements), a clustering algorithm could produce geometrically excellent (high-silhouette) clusters that nonetheless split or merge the true classes incorrectly (low purity) — because the clusters are internally cohesive but simply do not correspond to the semantic ground-truth groups.

Q3.5 [Research/complexity-oriented] Explain why naive hierarchical clustering costs $O(N^3)$ and describe the key data-structure idea that reduces this to $O(N^2 \log N)$.

Answer: A naive implementation performs $N-1$ merge steps (agglomerative); at each step it must scan the full $O(N^2)$ -sized distance matrix to find the globally closest pair of clusters to merge, giving $O(N^2)$ work per step $\times O(N)$ steps = **$O(N^3)$** total. The reduction to $O(N^2 \log N)$ comes from maintaining, for each cluster, a **priority queue (heap) of its nearest neighbor and distance**; finding the global minimum across all clusters is then an $O(\log N)$ heap operation rather than an $O(N^2)$ full re-scan, and only clusters whose nearest neighbor changed after a merge need their heap entry updated ($O(N)$ such updates per merge, each an $O(\log N)$ heap operation) — giving $O(N)$ merges $\times O(N \log N)$ work = $O(N^2 \log N)$ overall. The special case of **single linkage** can be computed even faster, in $O(N^2)$ time, via the SLINK algorithm, which exploits the fact that single-linkage merge distances relate directly to a minimum spanning tree of the distance graph.

Q3.6 Why does DBSCAN not require the number of clusters k as a parameter, yet it is still considered sensitive to parameter choice? Name the two parameters and describe a data property that makes DBSCAN struggle.

Answer: DBSCAN discovers the number of clusters as an *emergent* property of density-connectivity — it grows clusters outward from core points until density drops below the $(\epsilon, \text{minPts})$ threshold, so the count of resulting clusters is not fixed in advance. However, it is highly sensitive to its two parameters — ϵ (neighborhood radius) and **minPts** (minimum neighbors to qualify as core) — because both are *global, single-valued* thresholds applied uniformly across the whole dataset. DBSCAN therefore struggles with datasets containing **clusters of substantially different densities**: an ϵ tuned to correctly resolve a sparse cluster will merge points in a dense cluster into one blob (or vice versa) — a single global ϵ cannot simultaneously suit both densities. (This limitation directly motivated the OPTICS algorithm, which produces a reachability plot usable across a range of ϵ values rather than committing to one.)

Q3.7 A colleague proposes evaluating k-Means solely by minimizing E (the sum-of-squares objective) across different k , always picking the k with the smallest E . Critique this proposal.

Answer: E is **monotonically non-increasing in k** — increasing k can only help or leave E unchanged (in the extreme, $k=n$ gives $E=0$ with every point its own cluster). Minimizing E alone therefore always favors the largest possible k and never yields a meaningful "best" number of clusters — it provides no penalty for model complexity/overfitting. This is exactly why the Elbow method looks for a qualitative bend (diminishing marginal returns) rather than the numerical minimum, and why the Silhouette method (which explicitly trades off compactness *against* separation from other clusters) is preferred — its score can genuinely decrease when k gets too large, because splitting a natural cluster further reduces the separation term b_i without proportionally improving a_i .

Q3.8 Explain the practical meaning of the Hopkins statistic and why it should logically be checked *before*, not *after*, running k-Means or DBSCAN.

Answer: The Hopkins statistic tests the null hypothesis that the data was generated by a uniform random process (no true cluster structure) by comparing nearest-neighbor distances of actual data points against nearest-neighbor distances of artificially inserted uniform random points, over the same region. A value near 0.5 indicates the data is indistinguishable from uniform noise (no clustering tendency); values closer to 1 indicate strong non-random, clusterable structure. It should be checked *first* because clustering algorithms (k-Means especially) will **always output some partition** even when applied to perfectly random data — the algorithm has no internal mechanism to report "there was nothing here to cluster." Checking clustering tendency first avoids over-interpreting spurious structure and wasting effort tuning k or $(\epsilon, \text{minPts})$ on data that has no genuine clusters.

Comprehensive Exam & Research-Oriented Question Bank

The following 24 questions integrate material across all three parts, mirroring the style of TU Dresden MS-CS written exams (definition recall + calculation + conceptual application + open research framing). Model answers are concise but complete — expand with your own worked numbers when practicing.

R1 Contrast the "learning" (Mitchell) and "probabilistic" (Murphy) definitions of machine learning. Which one more naturally motivates treating clustering as "fitting $p(x)$ "?

Mitchell's is behavioral (T, P, E); Murphy's is representational (unknowns as random variables with distributions). The probabilistic view directly frames unsupervised learning as estimating an **unconditional density $p(x)$** that could generate the observed data — this is the natural bridge to mixture-model clustering (e.g. Gaussian Mixture Models), where each cluster corresponds to a mixture component.

R2 Explain, with a numeric example, why simple attribute matching similarity for categorical data ($S=1$ if equal, 0 otherwise) can be misleading, and how inverse occurrence frequency fixes this.

Simple matching treats a match on a common attribute value (e.g., "Adult=1", occurring in 90% of records) identically to a match on a rare value (e.g., a rare surname). But matching on the rare value is far more informative about true similarity — plausibly the two records are related — whereas matching on the common value carries little discriminative information. Inverse occurrence frequency scales the match's contribution by $1/p_k(x)^2$, so rare-value matches score much higher, mirroring the intuition behind IDF (inverse document frequency) in text retrieval.

R3 Given the Iris dataset's dendrogram computed via single vs. complete linkage produce different groupings — argue whether hierarchical clustering can be considered a single "algorithm" or a "family" of algorithms, and why this matters for reproducibility in research.

Hierarchical clustering is best understood as a **family**: the only fixed part is "iteratively merge/split based on a distance-between-clusters criterion"; the linkage function (single/complete/average/Ward, etc.) and the base point-distance metric are both free design choices that materially change the output. For reproducibility, a research paper must report *both* the point-distance metric *and* the linkage criterion — reporting only "we used hierarchical clustering" is insufficiently specified to reproduce results.

R4 Compute the Manhattan, Euclidean, and Maximum distance between $x_2=(3,5)$ and $x_3=(2,0)$.

$|\Delta_1|=1$, $|\Delta_2|=5$. Manhattan = $1+5 = 6$. Euclidean = $\sqrt{1+25} = \sqrt{26} \approx 5.10$. Maximum = $\max(1,5) = 5$. (Matches the reference matrices in §2.3 exactly.)

R5 A cluster assessment shows a high silhouette score but a low Hopkins statistic (~ 0.5). How do you reconcile this and what should you conclude about the clustering?

A Hopkins statistic near 0.5 suggests the underlying data has **no genuine non-random structure** — it looks statistically like uniform noise. Yet a clustering algorithm forced to partition this data into k groups can still produce geometrically "clean-looking" (high silhouette) partitions purely because any sufficiently regular partition of even random data tends to look locally compact/separated for a suitably chosen k . Conclusion: **internal validity metrics like silhouette cannot substitute for a clustering-tendency test** — a high silhouette score does not certify that meaningful clusters exist; it only certifies that the chosen partition is geometrically self-consistent given whatever data was fed in.

R6 Why is DTW necessary for time-series similarity even when both series already have equal length?

Equal length does not imply **temporal alignment**. Two series of the same length can still be "out of phase" — e.g., one signal lags the other by a few samples, or one is locally stretched/compressed relative to the other (a person speaking a word slightly faster). A direct index-wise L_p comparison would compare misaligned points and report large distances even for genuinely similar shapes. DTW re-aligns (warps) the series along the time axis before comparing, so length equality alone is not a substitute for DTW when phase/rate differences exist.

R7 Explain the computational-complexity argument for why the graph-isomorphism problem being "NP-hard" (as stated in the lecture) is somewhat imprecise, and why it still matters practically.

Graph isomorphism is **not known to be NP-complete**; complexity theory places it in a special class believed to be strictly between P and NP-complete (it is in NP, and in 2015 Babai showed a quasi-polynomial time algorithm exists, ruling out NP-completeness under standard assumptions). The lecture's "NP-hard" framing is a simplification of "no known efficient (polynomial) general algorithm." Practically the distinction rarely matters: for the graph sizes seen in real applications, exact isomorphism testing is still expensive enough that approximate similarity measures (max common subgraph, graph-edit distance) are used instead.

R8 You must cluster a dataset with $n=500$ points but $d=200,000$ dimensions (e.g., gene expression). Explain, referencing the curse of dimensionality, why applying k-Means with Euclidean distance directly is likely to fail, and propose a concrete alternative pipeline.

With $n \ll d$, pairwise Euclidean distances concentrate — nearly all points appear roughly equidistant, so k-Means' nearest-centroid assignment step becomes close to arbitrary/noisy, and its objective E provides little discriminative signal between candidate partitions. **Proposed pipeline:** (1) dimensionality reduction (e.g., PCA to retain the top variance-explaining components, or feature selection based on domain knowledge) to project into a lower-dimensional subspace where distances remain meaningful; (2) optionally, use a distance metric more robust in high dimensions (e.g., cosine similarity, or fractional L_p norms with $p < 1$); (3) then apply k-Means, k-Medoids, or density-based clustering in the reduced space; (4) validate with silhouette score and Hopkins statistic in the reduced space before trusting the result.

R9 Contrast "exclusive" and "overlapping" clustering with a concrete document-clustering example, and state which clustering algorithms from this course naturally produce which kind.

Exclusive: each document assigned to exactly one topic cluster (typical of k-Means, k-Medoids, AGNES/DIANA cut at a fixed level, and DBSCAN, which additionally allows the special "no cluster" / noise label). Overlapping: a document discussing both "finance" and "technology" belongs partially to both clusters — none of the base algorithms in this lecture natively support this; it requires extensions such as fuzzy c-means (soft cluster membership) or topic models (e.g., LDA) that are outside this lecture's scope but conceptually motivated by the "overlapping clustering" design axis introduced in §3.2.

R10 Given $\text{minPts}=4$ and a candidate core point with exactly 3 other points within ϵ (4 total including itself), classify the point and justify using the DBSCAN definition.

A core point requires **at least minPts points (including itself) within ϵ** . Here the point has 3 neighbors + itself = 4 points total, which equals $\text{minPts}=4$, satisfying "at least minPts ." **Classification: core point.** (Common exam trap: forgetting that the point itself is included in its own neighborhood count.)

R11 What is the key structural difference between the "Elbow method" and "Silhouette method" in terms of what geometric information each one uses?

The Elbow method uses only **intra-cluster** information (E , the sum of squared distances within clusters) — it never explicitly measures how far apart different clusters are, so its signal for "the right k " is indirect (diminishing marginal returns). The Silhouette method explicitly incorporates **both** intra-cluster compactness (a_i) and inter-cluster separation (b_i — distance to the nearest other cluster) in every single per-point score, making it a more complete (and computationally more expensive, since it needs distances to the nearest other cluster for every point) signal for choosing k .

R12 [Research] Propose and justify a modification to standard k-Means that would make it more robust to the "concentric rings" failure case discussed in Q3.2, without switching to a fully different algorithm family (i.e., stay within centroid-based partitioning).

One approach: apply a **kernel trick** (Kernel k-Means), implicitly mapping points into a higher-dimensional feature space via a kernel function (e.g. RBF kernel) in which the rings become linearly/convexly separable, then run standard centroid-based k-Means in that induced feature space. This preserves the "partitioning around means" structure of k-Means while escaping its Euclidean-convexity limitation — at the cost of needing to choose an appropriate kernel and its bandwidth parameter, analogous to choosing ϵ in DBSCAN.

R13 Explain why cosine similarity is a similarity measure but not, in its raw form, a metric distance — and describe how one converts it into a proper distance if a metric is required (e.g. for use inside DBSCAN, which needs a triangle-inequality-respecting notion of neighborhood).

Cosine similarity ranges in $[-1, 1]$ (or $[0, 1]$ for non-negative vectors) with *higher* meaning more similar — the opposite convention of a distance, and it does not directly satisfy distance axioms (e.g. it is not naturally 0 for identical vectors). A common conversion is the **cosine distance**, $d = 1 - \cos(x, y)$, which does satisfy $d=0$ for identical direction vectors and is commonly (though not universally, depending on normalization) treated as a pseudo-metric for use in neighborhood-based algorithms like DBSCAN; strictly speaking one should verify the triangle inequality holds for the specific normalized data before relying on formal metric-space guarantees.

R14 Why does the lecture emphasize that "Manhattan, Euclidean, and Maximum" distance calculations must all be shown as full distance matrices rather than a single pairwise number in exam solutions?

Exam grading for clustering problems (e.g., a full k-Medoids or AGNES trace) requires the *entire* pairwise distance structure, since every clustering algorithm's next merge/assignment step depends on comparing many pairs simultaneously — a single distance value cannot demonstrate whether a student correctly identifies the globally nearest pair, the correct linkage-based cluster distance, or performs a full DBSCAN neighborhood query. Presenting the full lower-triangular matrix is the standard way to demonstrate — and be graded on — complete, methodical computation.

R15 A student claims: "Since DBSCAN doesn't need k specified, it is strictly superior to k-Means." Critically evaluate this claim.

False as a blanket statement. DBSCAN trades the "specify k" burden for the arguably harder "correctly tune ϵ and minPts, possibly per local density region" burden, and it struggles badly with varying-density data (§3.9/Q3.6) where k-Means (given a reasonable k) may still produce a usable, if imperfect, partition. Additionally, k-Means is far cheaper computationally on large, high-dimensional, well-separated globular data, and its output (a fixed k with hard assignments and interpretable centroids) is often easier to act on downstream (e.g. for customer segmentation) than DBSCAN's variable number of clusters plus a noise category. The correct framing is that the two algorithms encode different *assumptions* about cluster shape and should be chosen based on which assumption better matches the data — not a strict superiority ordering.

R16 Formally state the medoid definition and explain why computing it is more expensive than computing a centroid.

medoid = $\operatorname{argmin}_{y \in X} \sum_{i=1}^n \operatorname{dist}(y, x_i)$ — the actual data point minimizing the total distance to all other points in the cluster. Computing a centroid requires one pass over the data, summing coordinates and dividing by n: $O(n \cdot d)$. Computing an exact medoid requires, for every candidate point y in the cluster, summing its distance to every other point: $O(n^2 \cdot d)$ in the naive case — quadratically more expensive, which is why PAM's full-medoid-search step is a major scalability bottleneck addressed by later variants (e.g. CLARA, CLARANS) that sample candidate medoids rather than checking all of them.

R17 Why might a research paper report both "purity" and "the Jaccard Index" for the same clustering result rather than just one?

Purity can be trivially maximized by making every cluster tiny singleton (each cluster then contains only one true class, giving purity=1) — it has a degenerate failure mode that rewards over-fragmentation and does not penalize the total number of clusters. The Jaccard Index (and complementary measures like the Rand Index) more directly measures overlap between the entire predicted partition and the true partition, penalizing both under- and over-fragmentation. Reporting both gives a fuller picture: high purity with low Jaccard would flag exactly this over-fragmentation failure mode.

R18 Explain how "reducing access to data points" (a big-data speed-up strategy from §3.10) could bias a k-Means result, and how one would mitigate that bias.

If a random sample is used instead of the full dataset, small or rare sub-populations may be under-represented or entirely missed in the sample, causing the fitted centroids to systematically ignore or merge minority clusters — a sampling bias. Mitigation: use **stratified** or **density-aware** sampling rather than uniform random sampling (ensuring rarer regions are still represented), increase the sample size for a sensitivity check, or run a final refinement pass over the full dataset once approximate centroids are found via the sample (a "coreset" approach).

R19 Using the LCSS example (xmjyauz / mzjawxu $\rightarrow$ LCSS = mjau), explain why LCSS can be more appropriate than edit distance for noisy time-series or sequence matching where insertions of irrelevant "noise" elements are common.

Edit distance penalizes every insertion/deletion with an explicit cost, so a sequence with many spurious noise insertions accumulates a large edit distance even if the "real" underlying pattern is fully present and in order. LCSS instead simply asks for the longest ordered common subsequence, effectively **ignoring** any extra noise elements at zero cost (they are just skipped over) — making it naturally robust to noise insertions/outliers as long as the true signal elements still appear in the correct relative order.

R20 [Research] The lecture states that "high-dimensional clustering often operates in subspaces rather than full space." Explain what problem this solves and name a challenge this introduces.

Full-space clustering in very high dimensions suffers the curse of dimensionality (§2.9) — distances become uninformative when computed across all d dimensions simultaneously, especially if only a subset of dimensions is actually relevant to the true cluster structure (irrelevant/noisy dimensions dilute the distance signal). Subspace clustering restricts distance computation to a relevant dimension subset (possibly a *different* subset per cluster), recovering a meaningful notion of proximity. **Challenge introduced:** the search space of possible dimension subsets is combinatorially large (2^d subsets), so subspace clustering algorithms (e.g. CLIQUE, a grid-based method mentioned in §3.4) must use heuristics (e.g. monotonicity/Apriori-like pruning: a subspace can only be dense if all its lower-dimensional projections are also dense) to make the search tractable.

R21 Justify, from first principles, why the sum-of-squares objective E used by k-Means implicitly assumes clusters are roughly spherical/isotropic in the original feature space.

E sums squared **Euclidean** distances to each cluster's centroid. The Euclidean distance treats all directions/dimensions equivalently (its level sets, per Fig. 2.1, are circles/spheres). Minimizing E therefore implicitly favors partitions whose clusters are compact in every direction equally — i.e., roughly spherical, similarly-scaled clusters. If true clusters are elongated or have very different variances along different axes (e.g. one feature measured in millimeters, another in kilometers, without standardization), Euclidean k-Means will distort or incorrectly split/merge them; this is why feature standardization (z-scoring) before k-Means is standard practice, and why algorithms like Gaussian Mixture Models with full covariance matrices generalize k-Means to handle elliptical clusters.

R22 Contrast the informational content carried by a single Hopkins-statistic value versus a full ODI (ordered dissimilarity image) heatmap for assessing clustering tendency.

The Hopkins statistic collapses all clustering-tendency information into a **single scalar** in $[0,1]$ — convenient for a quick automated check but silent about *where*, *how many*, or *what shape* potential clusters might have. The ODI heatmap (reordering the dissimilarity matrix and visualizing it) preserves much richer structural information — visible block-diagonal patterns reveal the approximate number and relative sizes of candidate clusters, and irregular/smeared blocks can hint at overlapping or elongated cluster shapes — at the cost of requiring visual/subjective human interpretation rather than a single automated threshold.

R23 Design a short experiment (data + procedure) that would empirically demonstrate the curse-of-dimensionality distance-concentration effect described in §2.9, and state what result you would expect to observe.

Procedure: sample n points uniformly at random in $[0,1]^d$ for a range of dimensions $d \in \{1, 2, 5, 10, 50, 100, 500\}$; for each d , compute all pairwise Euclidean distances and record the ratio $(\max \text{ distance} - \min \text{ distance}) / \text{mean distance}$, or equivalently the coefficient of variation (std/mean) of the distance distribution. **Expected result:** this ratio should shrink toward 0 as d increases — exactly the pattern shown in Fig. 2.2 — empirically confirming that in high dimensions nearly all pairs of points become approximately equidistant, undermining the discriminative power of any nearest-neighbor-based method (including k-Means, k-Medoids, and DBSCAN).

R24 Summarize, in one integrated paragraph suitable as a final exam "big picture" answer, how similarity/distance measures (Part II) determine the behavior of every clustering method covered (Part III).

Every clustering algorithm in this course is ultimately a search procedure layered on top of a chosen (dis)similarity function: k-Means/k-Medoids minimize a sum of (typically Euclidean) distances to representative points and therefore implicitly assume compact, roughly spherical clusters under that metric; AGNES/DIANA require both a point-level distance *and* a cluster-level linkage function, and different linkage choices (single vs. complete vs. average) applied to the identical point-distance matrix provably yield different dendrograms; DBSCAN redefines "cluster" entirely in terms of density-reachability under a distance-based ϵ -neighborhood, escaping the convexity assumption of centroid methods but inheriting sensitivity to how that neighborhood radius is chosen. Because the underlying distance/similarity function encodes the analyst's assumptions about what "similar" means for the application domain (Aggarwal's warning in §2.1), a poor choice of metric — or failure to standardize features, or ignoring the curse of dimensionality in high-d data — propagates directly into every downstream clustering result, regardless of which algorithm family is used; this is why Part II is a logical prerequisite to, not a separate topic from, Part III.

Cheat Sheet — Formulas, Algorithms & Comparisons

5.1 Distance / Similarity Formula Reference

Name	Formula	Use case
Manhattan (L1)	$\sum x_i - y_i $	Grid-constrained movement; robust per-axis
Euclidean (L2)	$\sqrt{\sum (x_i - y_i)^2}$	Default; used inside k-Means centroid computation
Maximum (L ∞)	$\max x_i - y_i $	Worst-axis-dominant applications (e.g. tolerance checks)
Simple Matching (categorical)	$S=1$ if equal else 0	Nominal attributes, no rarity weighting
Inverse Occurrence Frequency	$S=1/p_k(x)$ if equal else 0	Nominal attributes, rarer matches score higher
Cosine Similarity	$(x \cdot y) / (\ x\ \ y\)$	Text/document vectors; length-invariant
Edit (Levenshtein) Distance	min total cost of ins/del/sub	String/sequence alignment
LCSS	longest ordered common subsequence	Noise-robust sequence matching
DTW	Lp distance after time-axis warping/alignment	Time series with phase/rate shifts

5.2 Clustering Algorithm Comparison Matrix

Algorithm	Family	Needs k?	Cluster shape	Outlier handling	Complexity
k-Means	Partitioning	Yes	Convex / spherical	Poor (mean is pulled by outliers)	$O(n \cdot k \cdot d \cdot \text{iterations})$
k-Medoids / PAM	Partitioning	Yes	Convex / spherical	Robust (medoid is an actual point)	$O(n^2 \cdot d)$ per iteration (naive)
AGNES	Hierarchical (bottom-up)	No (cut hierarchy)	Depends on linkage	Single-link sensitive; complete-link robust	$O(N^3) \rightarrow O(N^2 \log N)$; $O(N^2)$ single-link
DIANA	Hierarchical (top-down)	No (cut hierarchy)	Depends on split criterion	Similar to AGNES	Generally more expensive than AGNES
DBSCAN	Density-based	No (emerges from ϵ , minPts)	Arbitrary	Explicit noise label	$O(n \log n)$ with spatial index; $O(n^2)$ naive

5.3 Cluster Assessment Toolkit

Question	Tool
Is there any clustering structure at all?	Hopkins Statistic; ODI heatmap
What is a good k? (no ground truth)	Elbow Method (intra-cluster only); Silhouette Method (intra + inter)
How good is the clustering vs. known labels?	Purity; Jaccard Index; F-score / Confusion Matrix
Does it make sense qualitatively?	Manual (expert) evaluation
Does it help the end application?	Indirect evaluation (downstream task performance)

5.4 Key Numbers & Facts to Memorize

- Silhouette coefficient range: **[-1, +1]**; higher is better.
- k-Means objective E: monotonically **non-increasing** in k — never use raw E to pick k.
- General hierarchical clustering: **$O(N^3)$** , reducible to **$O(N^2 \log N)$** ; single-link special case: **$O(N^2)$** .
- k-Means's underlying combinatorial optimization is **NP-hard**; Lloyd's algorithm is a heuristic converging only to a local optimum.
- Graph isomorphism: no known efficient general algorithm ("NP-hard" per the lecture); DBSCAN core-point rule counts the point **itself** among its neighbors.
- $L_\infty \leq L_2 \leq L_1$ always, for any pair of points.

